



南京大學

本科畢業設計

院 系 软件学院

专 业 软件工程

题 目 软件成分分析系统 Go、

Python 组件依赖关系分析模

块的设计与实现

年 级 2020 学 号 201250063

学生姓名 靳琦清

指导教师 刘峰 职 称 高级工程师

提交日期 2024 年 5 月 30 日



南京大学本科毕业论文（设计） 诚信承诺书

本人郑重承诺：所呈交的毕业论文（设计）（题目：软件成分分析系统 Go、Python 组件依赖关系分析模块的设计与实现）是在指导教师的指导下严格按照学校和院系有关规定由本人独立完成的。本毕业论文（设计）中引用他人观点及参考资源的内容均已标注引用，如出现侵犯他人知识产权的行为，由本人承担相应法律责任。本人承诺不存在抄袭、伪造、篡改、代写、买卖毕业论文（设计）等违纪行为。

作者签名：靳琦清

学号：201250063

日期：2024年5月8日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：软件成分分析系统 Go、Python 组件依赖关系分析模块的设计与实现

院系：软件学院

专业：软件工程

本科生姓名：靳琦清

指导教师（姓名、职称）：刘峰 高级工程师

摘要：

随着软件供应链的不断复杂化，软件成分分析的重要性日益突出。现代软件开发过程中广泛使用第三方库、开源组件和框架，这些元素在加速开发和提高效率的同时，也带来了潜在的安全和合规风险。为了应对当前开源发展趋势和复杂的供应链环境，本文提出并实现了一种轻量级的软件成分分析系统。该系统通过识别、跟踪和分析软件项目中的各种成分，帮助开发团队及时发现并修复安全漏洞，并确保项目符合相关法规和许可要求。

本文首先介绍了系统的技术栈以及整体架构，详细描述了系统的设计理念和目标。然后，通过用例图、系统顺序图和数据库设计等方式，本文重点阐述了 Go 和 Python 组件依赖关系分析模块的需求分析与概要设计。接着，借助流程图和逻辑代码，本文深入探讨了 Go 和 Python 组件依赖关系分析模块的详细设计与实现，包括依赖关系分析、组件信息爬取、SBOM 导出和 Excel 导出功能。最后，本文对系统的模块进行了全面的测试。

关键词：软件成分分析；依赖关系分析；软件供应链安全

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: Design and Implementation of Component Dependency Analysis (Go, Python) Module in SCA System

DEPARTMENT: Software Institute

SPECIALIZATION: Software Engineering

UNDERGRADUATE: Qiqing Jin

MENTOR: Feng Liu, Senior Engineer

ABSTRACT:

With the increasing complexity of the software supply chain, the importance of software composition analysis is becoming increasingly prominent. In modern software development, third-party libraries, open source components, and frameworks are widely used, which not only accelerate development and improve efficiency, but also bring potential security and compliance risks. In order to cope with the current trend of open source development and the complex supply chain environment, this thesis proposes and implements a lightweight software composition analysis system. This system helps development teams discover and fix security vulnerabilities in a timely manner by identifying, tracking, and analyzing various components in software projects, and ensure that the project complies with relevant regulations and licensing requirements.

This thesis first introduces the technology stack and overall architecture of the system, and provides a detailed description of the system's design philosophy and objectives. Then, through use case diagrams, system sequence diagrams, and database design, this thesis focuses on the requirement analysis and summary design of the Go and Python component dependency analysis modules. Next, with the help of flowcharts and logical code, this thesis delves into the detailed design and implementation of the Go and Python component dependency analysis module, including dependency analysis, component information crawling, SBOM and Excel export functions. Finally, this thesis conducted a comprehensive test of the modules in system.

KEYWORDS: Software Composition Analysis; Dependency Relationship Analysis;
Software Supply Chain Security

目 录

第一章 引言	1
1.1 项目背景	1
1.2 国内外研究现状及分析	2
1.3 本文主要工作	3
1.4 论文的组织结构	4
第二章 技术概述	5
2.1 Vue	5
2.2 Spring 和 Springboot	6
2.2.1 Spring	6
2.2.2 Springboot	7
2.3 Redis 和 PostgreSQL	8
2.3.1 Redis	8
2.3.2 PostgreSQL	8
2.4 其他相关技术	9
2.4.1 SCA 检测技术	9
2.4.2 Python 依赖管理与 Python 仓库	11
2.4.3 Go 依赖管理与 Go 仓库	12
2.4.4 软件物料清单	12
2.5 本章小结	12
第三章 系统概述和模块需求分析与概要设计	13
3.1 系统整体概述	13
3.1.1 系统业务说明	13
3.1.2 面向用户	13

3.1.3	系统功能概述	14
3.2	Go、Python 组件依赖关系分析模块需求分析	15
3.2.1	用例图和用例描述	15
3.2.2	模块顺序图	20
3.2.3	非功能需求描述	22
3.2.4	约束	22
3.3	Go、Python 组件依赖关系分析模块开发总体视图	22
3.4	Go、Python 组件依赖关系分析模块数据库设计	23
3.4.1	主要实体和实体关系	23
3.4.2	表结构设计	24
3.5	本章小结	29
第四章 Go、Python 组件依赖关系分析模块的详细设计与实现		31
4.1	Go、Python 组件依赖关系分析方法的设计与实现	31
4.1.1	Go、Python 组件依赖关系分析方法概述	31
4.1.2	Go、Python 组件依赖关系分析方法详细设计	31
4.1.3	Go、Python 组件依赖关系分析方法实现	34
4.2	Go、Python 组件爬取方法的设计与实现	40
4.2.1	Go、Python 组件爬取方法概述	40
4.2.2	Go、Python 组件爬取方法详细设计	41
4.2.3	Go、Python 组件爬取方法实现	42
4.3	SBOM 与 Excel 导出功能的设计与实现	45
4.3.1	SBOM 与 Excel 导出功能概述	45
4.3.2	SBOM 与 Excel 导出功能详细设计	45
4.3.3	SBOM 与 Excel 导出功能实现	47
4.4	本章小结	52
第五章 Go、Python 组件依赖关系分析模块测试		53
5.1	测试环境	53
5.2	功能测试	53
5.3	本章小结	55

第六章 总结与展望	57
6.1 总结	57
6.2 下一步工作	58
参考文献	61
致 谢	65

插图目录

2-1	Vue 响应式架构图	6
2-2	Spring 整体架构图	7
3-1	Go、Python 组件依赖关系分析模块用例图	15
3-2	Go、Python 组件依赖关系分析模块系统顺序图	21
3-3	Go、Python 组件依赖关系分析模块开发包图	23
3-4	Go、Python 组件依赖关系分析模块实体关系图	24
4-1	Go 组件依赖关系分析流程图	32
4-2	Python 组件依赖关系分析流程图	33
4-3	go mod graph 执行结果示例	34
4-4	解析 go mod graph 执行结果核心代码	35
4-5	由依赖树生成依赖表核心代码	36
4-6	调用 pipdeptree -json-tree 命令核心代码	37
4-7	pipdeptree -json-tree 执行结果示例	38
4-8	解析 pipdeptree -json-tree 执行结果核心代码	39
4-9	查看依赖树用户界面	40
4-10	查看平铺依赖表用户界面	40
4-11	组件爬取流程图	42
4-12	Go 组件爬取核心代码	43
4-13	获取 go 组件名单代码实现	44
4-14	获取 Python 组件名单代码实现	44
4-15	导出 SBOM 流程图	46
4-16	导出 Excel 流程图	47
4-17	底层应用导出 SBOM 核心代码	48

4-18 非底层应用导出 SBOM 核心代码	49
4-19 底层应用 SBOM 导出结果（部分）示例	50
4-20 非底层应用 SBOM 导出结果示例	50
4-21 导出 Excel 核心代码	51
4-22 导出 Excel（简略）结果示例	52

表格目录

3-1	系统用户描述	14
3-2	查看扫描结果用例	16
3-3	查看应用扫描与分析结果用例	17
3-4	查看组件详细信息用例	18
3-5	查看导出应用 SBOM 用例	19
3-6	查看导出应用 Excel 用例	20
3-7	系统的非功能需求表	22
3-8	plt_application 表	25
3-9	plt_app_component 表	26
3-10	plt_app_dependency_tree 表	26
3-11	plt_app_dependency_table 表	27
3-12	plt_go_component 表	27
3-13	plt_python_component 表	28
3-14	plt_visited_python_packages 表	29
5-1	测试环境表	53
5-2	功能测试表	53

第一章 引言

1.1 项目背景

在当今软件开发过程中，使用开源库和第三方组件已成为一种普遍的实践。Forrester 的报告指出，78% 的库是开源的，开源库中开源软件的数量按每年 21% 速度在增长^[1]。Veracode 的报告显示，10% 的应用使用了多余 1400 个库，90% 的应用使用了多余 34 个库，每个应用平均使用了 283 个库^[2]。在这些第三方库和组件中，不可避免地存在着各种潜在的漏洞和法律风险。Black Duck 审计团队在一份报告中指出，96% 的代码库涉及开源，84% 的代码库中存在漏洞，53% 的代码库中存在许可证冲突，31% 的代码库中没有许可证或使用自定义许可证^[3]。因此如何对软件进行全方面的检测和分析，就成了一个至关重要的问题。由此，软件成分分析技术应运而生，并在软件供应链管理中发挥着重要的作用。^[4]

软件成分分析（Software Composition Analysis, SCA）是 Gartner 定义的一种应用程序安全检测技术，该技术通过分析软件中所包含的一些信息或特征文件，来识别软件中所使用的各种源码、组件、模块、框架和库等关键成分信息，以及这些成分中所含有的各种已知漏洞、许可证和知识产权信息^[5]。进一步地，SCA 会帮助用户识别和管理软件中复杂的依赖关系，生成软件物料清单和项目报告，分析、响应与治理各种安全漏洞，确保许可合规性与兼容性要求，检测同源代码片段并评估知识产权风险，以确保软件供应链中组件的安全^[6]。SCA 的价值在于它所提供的安全性、速度和可靠性，仅靠手动跟踪开源代码根本无法处理数量庞大的开源组件。随着云原生应用和更复杂的应用日益普及，采用稳定可靠的 SCA 工具已经成为必然。

在 SCA 流程中，成分识别和依赖分析是关键的上游和基础环节，其旨在通过扫描源代码或二进制文件等方式，准确识别出软件项目中直接或间接使用的所有开源或商业的第三方库和组件，以及这些组件和软件项目之间的直接或间接依赖关系^[7]。通过成分识别和依赖分析，可以帮助开发团队全面了解软件项目

中的组件组成和依赖结构，识别其中的兼容性问题和过时组件，以及减少不必要的依赖，从而为开发者团队提供优化项目结构和提高软件可维护性的机会^[8]。因为软件成分和依赖信息的准确与完整，决定了后续对软件漏洞、许可证、知识产权等信息的检测与分析质量，所以可以说成分识别和依赖分析这一步骤是漏洞检测、许可合规性评估、风险管理等一系列 SCA 下游环节的基础。

1.2 国内外研究现状及分析

国外在 SCA 领域已经相当成熟，许多研究机构和企业推出了高质量的 SCA 解决方案，在检测技术、自动化程度和数据丰富性方面都表现出色。其中的代表性产品有 Synopsys 的 Black Duck、OWASP 的 Dependency-Track、Sonatype 的 Sonatype Platform、Mend.io 的 Mend SCA 等。

Black Duck 提供依赖分析、二进制、代码片段、签名分析、容器扫描的多方位全面扫描功能，能够有效管理应用、容器以及任何其他软件工件中使用开源所带来的安全性、许可合规性和代码质量风险。

Dependency-Track 通过利用软件物料清单 (SBOM) 的能力，采用独特且极具价值的方法，提供了传统 SCA 解决方案无法实现的功能，从而帮助组织识别和降低软件供应链中的风险。^[9]

国外的 SCA 研究和产品不断创新，并与 DevOps 等现代开发流程紧密结合。这种深度融合不仅提高了软件供应链管理的效率和可靠性，还使得 SCA 产品在持续集成和持续交付 (CI/CD) 环境中更具实用性和灵活性^[10]。此外，国外的 SCA 产品注重与其他安全工具和平台的整合，提供全面的“一站式”服务。这些供应商种类丰富，各有其专业领域和独特的优势产品^[11]。

国内的 SCA 技术研发和市场应用起步较国外稍晚，其技术起源也有不同。国外用户对知识产权的关注较高，因此 SCA 产品在代码审计、许可合规和兼容性方面表现突出^[12]；相对而言，国内用户更注重监管合规，包括对开源漏洞风险管理和软件可维护性。因此，当前国内 SCA 产品主要侧重于漏洞识别、修复和响应处理。不过虽然国内的 SCA 研究虽然起步较晚，但已经开始受到国内企业用户的广泛关注，近年来取得了一定的发展。目前国内的 SCA 技术代表性厂商有悬镜安全、墨菲安全、北大库博、思客云等。

OpenSCA 是悬镜安全旗下的开源软件成分分析（SCA）工具。该工具通过对软件成分、依赖项、特征、引用和合规性进行分析，深入挖掘组件中潜在的安全漏洞和开源协议风险，确保应用中引入的开源组件的安全。它的特点在于轻量且易于使用，功能全面，支持用户自定义配置漏洞库和私有库，适用于 IDE、命令行和云平台等各种场景，以及离线和在线模式。OpenSCA 能够灵活地融入开发流程，为企业、组织和个人用户提供透明的组件资产和风险清单。

墨菲安全旗下的苏木 SCA 拥有供应链资产识别管理、风险检测、安全控制、一键修复的完整供应链安全管理能力，包含多种检测模式来灵活适配多应用场景，使用独家专业漏洞知识库分析漏洞真实影响进行风险检测，并支持组件升级兼容性评估、一键修复编码阶段漏洞等多种功能。

总之国内供应商正在不断完善和创新 SCA 产品，以满足企业用户的需求。未来，随着国内 SCA 领域的发展和技术的进步，国内的研究和产品将进一步提升在全球范围内的竞争力。

1.3 本文主要工作

本文从当前软件行业的业务需求和背景出发，深入探讨了开源发展趋势下软件成分分析技术的重要性及其在国内外的研究现状。通过分析，本文明确了系统的研发目的、实际应用意义以及未来的研究方向，旨在为用户提供一套高效、准确的软件成分分析工具。

在 Go、Python 组件依赖关系分析模块的设计与实施方面，本文做了以下详细工作：

首先，在成分识别和依赖分析上，我们充分利用了 Go 和 Python 两种语言各自包管理器的强大功能。通过解析项目配置文件，我们能够准确获取项目的依赖信息以及它们之间的依赖关系。随后，我们对这些原始数据进行了清洗和整理，转化为易于管理和查询的知识库持久对象，为后续的分析 and 查询提供了坚实的数据基础。

其次，在组件信息的获取方面，我们利用 Go 和 Python 官方仓库提供的 API 接口，设计了一套高效的爬虫系统。该系统能够自动爬取组件的详细信息，包括名称、版本号、描述、地址、作者以及许可证等关键信息。当在依赖分析过程中

遇到知识库中缺失的组件信息时，爬虫系统能够实时进行爬取，并自动更新到知识库中。此外，我们还获取了一份包含大量 Go 和 Python 组件的详尽名单，通过定期或触发式的批量爬取，实现了知识库的增量更新和扩展。

最后，在软件物料清单（SBOM）的导出方面，我们支持了应用级别的导出功能，并特别考虑了多层次结构应用的 SBOM 导出形式。用户可以根据自己的需求，选择导出整个项目或特定模块的 SBOM 信息。此外还支持导出 Excel，方便在办公环境中进行查看和处理。这一功能的实现，在软件成分分析结果的可用性和可操作性方面有极大提升。

1.4 论文的组织结构

本文共分为六个章节，各章节内容如下：

第一章 引言：介绍了系统的背景和设计动因，概述了国内外软件成分分析领域的研究现状和成熟产品。同时，介绍了本文的主要工作，阐明了本文研究的目标和意义。

第二章 技术概要：描述了系统开发和运维所采用的主要技术栈，以及这些技术在系统中的具体作用。此外，还介绍了软件成分分析领域的三种主要检测技术，以及系统选择的检测技术及其选择原因。

第三章 系统概述和模块需求分析与概要设计：首先，对系统进行了整体概述，包括系统的业务说明、面向的用户和系统的核心功能。然后，对 Go、Python 组件依赖关系分析模块进行了需求分析，以及相关的非功能需求和约束条件。最后描述了模块的开发视图、主要实体及其关系与数据库设计。

第四章 Go、Python 组件依赖关系分析模块的详细设计与实现：深入探讨了 Go 和 Python 组件依赖关系分析模块中依赖关系分析、组件爬取方法和 SBOM、Excel 导出功能的详细设计和实现过程。

第五章 测试：详细介绍了系统的测试工作，包括测试的范围、方法和结果。

第六章 总结与展望：总结了本文的主要内容，并针对本系统的不足，确定了系统的下一步工作。

第二章 技术概述

2.1 Vue

Vue 是一款用于构建用户界面的 JavaScript 渐进式框架，以其易学、易用且灵活的特性而广受欢迎。它基于标准 HTML、CSS 和 JavaScript 构建，提供声明式、组件化的编程模型，可以帮助用户高效开发用户界面。^[13]

Vue 的两个核心功能为：

1. 声明式渲染：Vue 基于标准 HTML 扩展了一套模板语法，使开发者可以声明式地描述最终输出的 HTML 和 JavaScript 状态之间的关系。这一特性使开发者能够轻松地创建复杂的用户界面，并将数据与视图保持同步。通过模板中简单而直观的指令，如数据绑定、条件渲染、循环渲染等，开发者可以有效地管理和控制视图层的显示和行为。^[14]
2. 响应式：Vue 提供了一个强大的响应式系统，能够自动跟踪 JavaScript 状态的变化，并在其发生变化时实时更新 DOM。这意味着当数据发生变化时，界面会自动反映变化，而无需手动操作。这种自动化的更新机制不仅简化了开发流程，而且提高了应用的性能和用户体验。Vue 的响应式系统还支持复杂的数据结构，如嵌套对象和数组，并提供可观察的数据和计算属性，使开发者能够灵活地处理和管理应用的数据。其原理如图2-1所示。

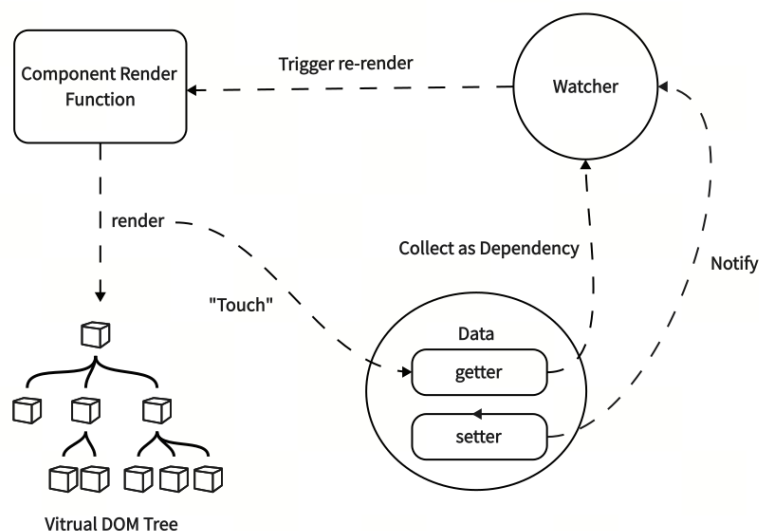


图 2-1 Vue 响应式架构图

本系统使用 Vue 来编写展示层，即前端用户界面。

2.2 Spring 和 Springboot

2.2.1 Spring

Spring 是一个广泛应用与 Java 开发的轻量级开源框架，旨在简化企业级应用程序的开发、部署和管理。它最初由 Rod Johnson 于 2002 年提出和发布，后来发展成为一个庞大的生态系统，涵盖了多个子项目和模块。

Spring 的核心是依赖注入（Dependency Injection, DI）和控制反转（Inversion of Control, IoC）的设计原则，它们使开发者能够更灵活地管理应用程序组件和依赖关系。通过使用 Spring，开发者可以专注于业务逻辑，而不是被基础设施代码所困扰。

Spring 提供了丰富的功能和工具，涵盖以下主要方面^[15]：

1. Spring MVC: 一个功能强大的 Web 框架，支持构建 RESTful API 和 Web 应用。
2. Spring Boot: 用于快速构建基于 Spring 的独立应用，提供自动配置和嵌入式服务器支持。
3. Spring Data: 简化数据访问和操作，支持多种数据库和数据存储技术。

4. **Spring Security:** 一个全面的安全框架，提供身份验证、授权和保护 Web 应用程序的能力。
5. **Spring Cloud:** 提供一系列工具和组件，支持微服务架构的开发和部署。

此外，Spring 生态系统还包含其他子项目，如 Spring Batch（批处理框架）、Spring Integration（企业集成框架）和 Spring AMQP（消息队列支持）等。通过提供全面的工具和模块，Spring 帮助开发者构建高质量、可扩展的应用程序，并支持其在多种领域的应用。

Spring 的整体架构如图2-2所示。

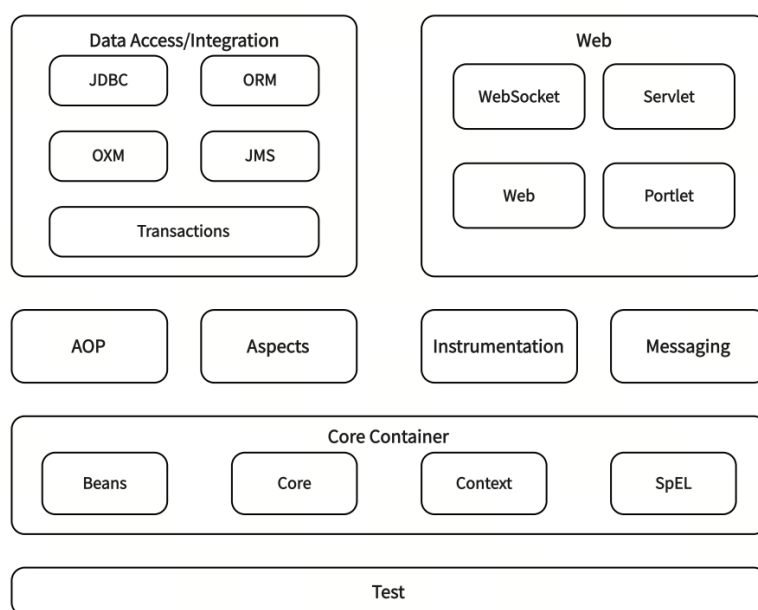


图 2-2 Spring 整体架构图

2.2.2 Springboot

Spring Boot 是 Spring 框架的一个子项目，旨在解决传统 Spring 应用程序开发中的配置复杂性问题。虽然 Spring 提供了丰富的功能和灵活性，但其配置和依赖管理对于新手和大型项目而言可能过于繁琐。Spring Boot 引入了自动配置、嵌入式服务器支持和预配置的启动器依赖项，简化了 Spring 应用程序的开发和部署流程。通过这些特性，开发者能够快速构建独立运行的 Spring 应用程序。此外，Spring Boot 提供生产级特性，如度量指标、健康检查和外部配置，以及命令行界面（CLI），进一步提升了应用程序的健壮性和开发效率。因此，Spring Boot

成为了开发和部署 Spring 应用程序的首选工具，为各种规模的项目提供了更简洁高效的解决方案。

本系统使用 Springboot 作为系统后端的整体框架。

2.3 Redis 和 PostgreSQL

2.3.1 Redis

Redis（Remote Dictionary Server）是一个开源的内存数据结构存储系统，被广泛用于缓存、持久性存储和分布式应用等多种场景中。Redis 以其极高的性能而著称，提供了丰富的数据类型，并支持事务、持久化、LUA 脚本、LRU 驱动事件、发布订阅、分布式等多种特性和功能。

本系统使用 Redis 来实现缓存与文件的上传与下载功能。

2.3.2 PostgreSQL

PostgreSQL 是一个功能强大的开源的对象-关系数据库管理系统¹，因其功能丰富、性能高和稳定性而广受欢迎。它最初由加州大学伯克利分校的研究项目开发，后来逐渐发展成为一个由社区驱动的项目。随着时间的推移，PostgreSQL 被誉为最先进的开源数据库之一，被广泛应用企业级应用、Web 应用、金融行业和地理信息系统等各种场景。^[16]

PostgreSQL 具有以下特点^[17]：

1. 函数: PostgreSQL 支持在多种语言中编写函数,包括 SQL、PL/pgSQL、Python 等。用户可以创建自定义函数来简化复杂的业务逻辑。函数还可以用于触发器和规则，在数据变化时自动执行特定操作，提高数据库的灵活性和自动化程度。
2. 索引: PostgreSQL 提供多种类型的索引，如 B-Tree、Hash、GIN、GIST、SP-GIST 等，支持并行索引构建和同时使用多个索引，用户也可以自定义索引。

¹ 对象关系数据库系统（ORDBMS）是面向对象技术与传统的关系数据库相结合的产物，其将所有实体都看成对象，并将这些对象类进行封装。

3. 触发器: PostgreSQL 支持触发器, 可以在特定的数据库操作 (如插入、更新、删除) 发生时执行特定的函数。PostgreSQL 提供了行级和语句级触发器, 适用于不同的应用场景。
4. 多版本并发控制 (MVCC): PostgreSQL 使用多版本并发控制 (MVCC) 来处理并发事务。MVCC 允许多个事务同时读取数据, 而不会互相干扰, 确保了数据库在高并发情况下的性能和稳定性。
5. 丰富的数据类型: PostgreSQL 支持多种内置数据类型, 包括基本类型 (如整数、浮点数、字符串) 和复杂类型 (如数组、JSON、XML)。用户也可以自定义数据类型, 以适应特定应用场景的需求。
6. 规则: PostgreSQL 支持规则, 一种类似于视图的机制, 可以在查询和修改数据时自动执行特定的操作。通过规则, 用户可以重写查询, 实现复杂的数据操作和数据变换。

本系统使用 PostgreSQL 数据库主要来完成两个工作。第一, 作为基础数据库, 持久化存储和管理用户信息、权限设定以及与系统功能相关的数据, 并支持高并发的访问需求。第二, 作为知识库, 存储最高千万级别的组件 (包括库、模块、包)、许可证和漏洞信息, 并支持高效的查询、更新、删改操作。

2.4 其他相关技术

2.4.1 SCA 检测技术

目前各 SCA 产品对于软件的成分分析, 主要有三种检测技术:

1. 基于包管理器 (构建) 的检测技术
2. 基于源码相似度匹配的检测技术
3. 基于二进制的开源软件识别技术

基于包管理器 (构建) 的检测技术 (又称特征检测) 是通过利用当前大多数编程语言具有的包管理器 (如 Java 中的 Maven 和 Gradle, Go 中的 mod, JavaScript 中的 npm, Python 中的 pip 和 conda 等) 来识别软件项目的第三方依赖项和传递依赖关系, 并构建项目的完整依赖树^[18]。使用此技术的 SCA 扫描有两种主要模式: Build Scan 和 Pre-build Scan。Build Scan 是指在具备项目完整编译环境的情

况下，通过包管理器的功能（如 `maven dependency:tree` 或 `go mod graph` 等）生成完整的依赖树，这样生成的依赖树与实际构建情况一致^[19]。**Pre-build Scan** 是在未构建项目之前进行的扫描，通过解析清单文件（如 `pom` 文件）来获取项目依赖信息，这意味着其能在项目的早期开发阶段就提供风险指导，但仅凭此无法发现项目的深层间接依赖，仍需借助知识库中的组件依赖信息和语言特定的依赖传递算法来还原项目的完整依赖树。

基于源码相似度匹配的检测技术是指通过将项目源代码片段与已知开源项目的代码片段进行比对，来识别项目中潜在的开源代码，以避免出现因抄袭第三方开源代码导致的合规风险。该技术主要包括两个步骤：一是对已知第三方库的代码创建代码指纹，目前最多的两种做法是将源码视作自然语言进行的构建和基于 **Token** 的构建；二是实现一个相似度匹配算法，常用的方法包括机器学习、神经网络、大模型等^[20]。

基于二进制的开源软件识别技术是指通过分析编译后的二进制文件和库，来识别项目中的开源组件。该技术依赖于二进制文件中的符号和字符串提取，具体分析过程通常涉及提取二进制文件中的函数名、常量字符串等信息。这种方法尤其适用于检测无法访问源代码的项目，例如固件和容器镜像，但缺点是检测的准确率一般不如前面两种高。^[21]

在本论文的设计与实现过程中，通过比对各种 **SCA** 检测技术的优劣势，最终确定了使用基于包管理器的检测技术中的 **Build Scan** 作为依赖管理模块的主要实现方案，这一选择基于以下优势：

1. **Build Scan** 实现简洁且高效。通过利用包管理器的功能，能够快速、准确地解析项目依赖信息。此方法确保依赖树与实际构建的项目保持一致，为依赖管理提供了准确的基础。
2. 尽管 **Build Scan** 仍然需要一个知识库来支持依赖检测，但与基于源码相似度扫描不同，其不需要完整的开源代码知识库。相反，可通过实时爬取和增量更新的方式维护知识库，从而降低了实现难度和数据获取的复杂性。
3. 与基于源码相似度扫描和基于二进制识别技术相比，**Build Scan** 能够获得完整的项目依赖树，而不仅仅是识别出项目中的组件，这使得依赖管理更加全面和精确。
4. 与 **Pre-build Scan** 相比，**Build Scan** 不需要实现复杂的特定于语言的依赖传

递算法，并且 Pre-build Scan 也只是在尽力还原 Build Scan 的结果。通过直接利用包管理器的功能，Build Scan 避免了这一风险，提高了整体的可靠性。

由于不同语言的包管理器、项目结构与源码特性不同，基于包管理器的 Build Scan 检测技术需要针对语言进行定制化的检测，同时不同语言的组件信息的字段也有所出入，本文主要实现了 Go、Python 两种语言的依赖管理。

2.4.2 Python 依赖管理与 Python 仓库

Python 语言中常用的包管理工具为 pip、conda、poetry。Pip 是 Python 官方的包管理工具，提供了对 Python 包的查找、下载、安装和卸载的功能，Python 2.7.9 或 Python 3.4 以上版本都自带 pip 工具，并且 pip 支持在任何环境下安装 python 包。Conda 是一个跨平台的包管理和环境管理工具，其包管理功能与 pip 类似，但仅支持在 conda 环境中安装包。Poetry 是目前较新的 Python 依赖管理和打包工具，提供了强大的依赖解析和项目管理功能，被认为是当前最为完善的 Python 依赖包管理解决方案之一。

一般的 Python 项目中，通常会使用 requirements.txt 和 setup.py 文件来定义依赖项。requirements.txt 是一个文本文件，用于详细记录项目所依赖的第三方库及其具体版本信息，它在项目开发和部署阶段为环境管理提供了参考。setup.py 文件是用于打包和发布 Python 项目的配置文件或脚本，它定义了项目的元数据（如名称、版本号、作者等），并通过 install_requires 参数指定项目的最低依赖关系。根据 Python 官方文档，install_requires（在 setup.py 中）明确了单个项目的依赖关系，而 requirements.txt 则通常用于定义完整 Python 环境的依赖项。但在大多数情况下，可能需要这两个文件同时存在，才能够正确管理包依赖和进行包的发布。^[22]

Pipdeptree 是 Python 开源库中的一款命令行工具，用于显示全局或特定环境中的依赖树，帮助开发者了解项目的依赖关系以及各个依赖库之间的关联。在安装后，可以通过执行 pipdeptree 命令来查看当前环境中的依赖树。该工具还提供了 -json 或 -json-tree 选项，允许将依赖树以 JSON 格式输出。此外，使用 -packages package_name 选项，可以查看特定包的依赖树。

PyPI(The Python Package Index) 是 Python 官方的包管理存储库,也是 Python 最主要的库源,其由 Warehouse 项目提供支持,共计维护了有大约 53 万个项目和 560 万个版本,涵盖绝大部分常用的 python 组件库。^[23]

2.4.3 Go 依赖管理与 Go 仓库

Go Mod (Go Modules) 是 Go 语言的模块化包管理工具,于 Go 1.11 版本引入,为 Go 项目提供了强大的依赖管理和模块化支持。通过 go.mod 文件,Go Mod 定义项目的模块名称和依赖信息,并使用语义化版本号管理模块版本。Go Mod 能自动解析依赖信息,生成项目的依赖树,并缓存已下载的依赖模块,以提高构建效率。此外,Go Mod 支持模块代理和镜像选择,帮助开发者加速模块下载。它还提供模块验证和修剪功能,确保项目依赖的完整性和简洁性。^[24]Go Mod 提供了 go mod graph 命令,通过分析 go.mod 文件中的依赖信息,输出一个以文本形式表示的依赖关系图,里面包含了项目的直接和间接依赖。

Go Package Index 是 Go 语言的官方包管理和文档平台,提供丰富的 Go 语言包和模块信息。开发者可以通过该平台搜索、查找和下载 Go 包,并查看详细的文档、API 参考、代码示例、版本历史和社区反馈。

2.4.4 软件物料清单

软件物料清单 (Software Bill of Materials, SBOM) 是一份详细记录了软件构建过程中使用的所有组件、库和依赖项的清单,其中的每个元素 (可能是一个组件) 会记录包括标识符、名称、版本、来源、许可证、依赖关系等详细的信息^[25]。SBOM 在软件供应链的透明度和可追溯性方面有着重要作用,能帮助开发团队更好地进行风险管理和安全保障工作,大部分的 SCA 工具都支持生成 SBOM。常见的 SBOM 标准格式有 SPDX、CycloneDX、SWID 等。

2.5 本章小结

本章主要介绍了软件成分分析系统 Go、Python 组件依赖关系分析模块所使用的技术栈,包括前端、后端框架、数据存储以及与 SCA 成分识别与依赖分析功能相关的技术,并阐述了每种技术的特点、使用缘由。

第三章 系统概述和模块需求分析与概要设计

3.1 系统整体概述

3.1.1 系统业务说明

本系统是一款由西门子（SIEMENS）实际业务需求出发，旨在提供软件供应链安全的轻量级软件成分分析系统。系统支持企业用户与权限管理、多层次结构并可以升级、锁定或发布应用的应用管理和包含开源、商用、闭源三种组件类型的组件管理，为用户提供全面的监控和管理，确保项目的合规性和安全性。系统支持 Java、JavaScript、Go、Python 四种语言，使用基于包管理器（构建）的 Build Scan 检测方法，通过分析应用的特征文件，扫描出其中的组件及其依赖关系。然后通过一个涵盖了规模庞大并支持增量更新的组件、许可证和漏洞的知识库来显示组件的详细信息以及应用中的许可证、漏洞信息，并且可以导出应用的 SBOM、Excel 和 Html 报告。借助本系统，用户可以有效识别和解决供应链中的潜在安全问题，确保软件项目的质量和稳定性。

3.1.2 面向用户

本系统主要面向的用户为企业中软件供应链的利益相关者与管理者，包括开发人员、软件架构师、安全团队、合规团队、运维人员、产品经理等。这些用户需要对软件项目中的开源组件和第三方库进行全面了解和管理，包括其版本、许可、依赖关系和潜在的安全漏洞，确保软件供应链的质量和合规性，确保项目在开发、部署和维护过程中符合安全和法律要求。此外，这些用户需要通过系统提供的报告和分析功能来优化决策过程，提高项目的效率和质量。

此外，本系统也面向正在开发软件项目的非企业个体用户，用以进行快速检测和评估他们所使用的开源软件和第三方库的安全性和合规性。个人开发者和小团队可以利用系统简便的界面和工具，快速识别所依赖的组件中的潜在风险

和漏洞。

表3-1描述了本系统所适用的用户。

表 3-1 系统用户描述

用户类别	描述
非企业个体用户	该类用户一般是个人开发者或小型团队，正在使用或开发小型软件项目，可能缺乏对软件供应链中开源组件和第三方库的全面了解和管理的资源。他们的主要需求是快速检测和评估所使用的开源软件和第三方库的安全性和合规性，以确保项目的稳定性和可靠性。本系统提供直观的界面和快速扫描功能，为用户提供即时反馈和建议，帮助他们及时修复问题，并避免潜在风险。此外，系统还能生成简洁的报告，帮助他们更好地理解和管理项目。
企业软件供应链的利益相关者或管理者	该类用户包括企业中的开发人员、软件架构师、安全团队、合规团队、运维人员和产品经理等企业软件供应链的利益相关者或管理者。他们在管理复杂的软件供应链时，需要面临对开源组件和第三方库的全面监控和管理的挑战。本系统提供实时监控和详细的报告，帮助用户识别、管理、追踪软件组件，确保项目在开发、部署和维护过程中符合安全和法律要求。此外，系统与 DevOps 流程紧密结合，支持持续集成和持续交付（CI/CD），从而提高团队协作，提升项目交付速度和质量。通过系统的强大功能，用户可以高效管理整个项目的供应链，提升项目的效率、质量和安全性。

3.1.3 系统功能概述

以下为本系统主要的功能概述：

- **用户与权限管理：**管理员或特定角色的用户可以为其他用户分配所属部门和角色，并且可以为应用添加成员以支持协作。不同角色拥有不同的权限。
- **应用与组件管理：**用户可以对应用与组件进行管理，包括新建、查询、更新、删改应用或组件信息（但不允许对开源组件进行修改）。此外，用户可以升级、锁定与发布应用，发布应用后会自动生成相应的组件。系统还支持导出应用的 SBOM、HTML 或 Excel 格式的报告。
- **依赖分析与管理：**用户可以上传扫描文件，系统扫描完成后，以依赖树或平铺依赖表的形式展示应用或组件的依赖信息，并可以生成不同版本之间

的依赖对比树。在扫描过程中，系统会识别出知识库中缺失的组件，并自动从公共仓库获取组件信息，对知识库进行增量更新。

- **许可证管理：**用户可以查看与搜索应用、组件或许可证库中的许可证，并获取许可证的详细信息。同时，系统支持检测应用或组件的许可证存在的冲突。
- **漏洞管理：**用户可以查看和搜索应用、组件或漏洞库中的漏洞信息，并查看漏洞的详细信息，以及获取相关漏洞的修复建议。

3.2 Go、Python 组件依赖关系分析模块需求分析

3.2.1 用例图和用例描述

如图3-1所示，Go、Python 组件依赖关系分析模块总共包含 5 个用例，分别为上传扫描文件、查看应用扫描与分析结果、查看组件详情、导出 SBOM、导出 Excel。

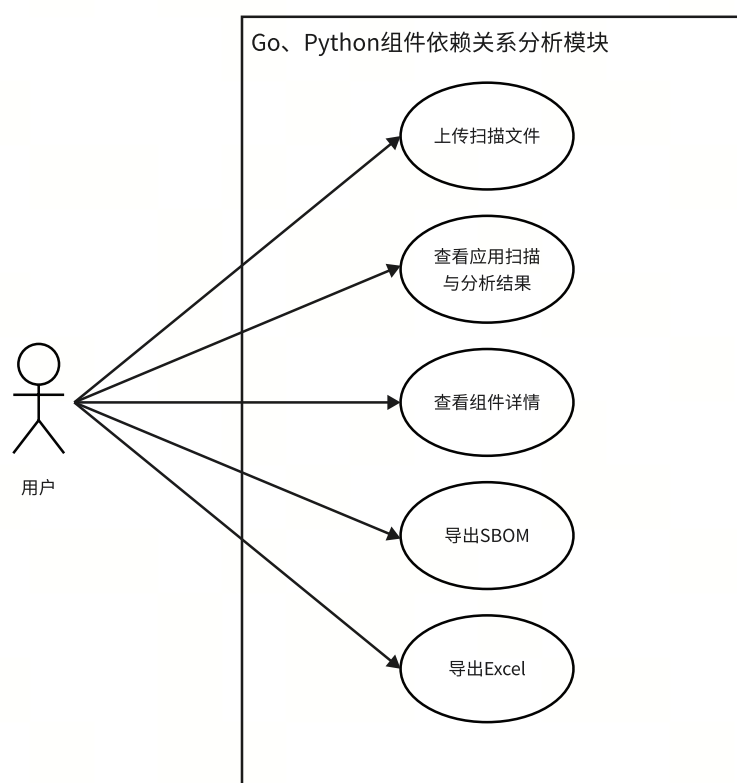


图 3-1 Go、Python 组件依赖关系分析模块用例图

表3-2描述了上传扫描文件用例，用户在新建完应用后或者在上传闭源组件

时，可以上传扫描文件进行扫描和分析，扫描失败还可以重新扫描。

表 3-2 查看扫描结果用例

名称	内容描述
ID	01
用例名称	上传扫描文件
参与者	用户
用例描述	分析应用或上传闭源组件时，用户需要按语言和工具上传扫描文件
触发条件	应用详情或上传闭源组件界面，用户点击上传扫描文件
前置条件	1. 系统及其所处网络环境正常运作 2. 用户已正确登录系统且拥有相关权限 3. 应用已正确创建
后置条件	系统返回状态码并存入数据库
基本流程	1. 用户点击上传扫描文件或重新扫描 2. 系统展示选择语言表单 3. 用户选择语言 4. 系统根据语言展示选择工具表单 5. 用户选择工具 6. 系统根据语言和工具展示上传文件表单 7. 用户上传文件并确认 8. 系统返回提交结果
扩展流程	7a. 用户上传的文件不符合语言和工具要求的规范，系统返回错误提示 8a. 如果扫描成功，则系统显示扫描成功，并生成对应依赖信息 8b. 如果扫描失败，则系统显示扫描失败，用户可以重新扫描，进入步骤 1

表3-3描述了查看应用分析结果用例，在扫描成功后，用户可以查看扫描与分析结果，包含依赖树与平铺依赖表等信息。

表 3-3 查看应用扫描与分析结果用例

名称	内容描述
ID	02
用例名称	查看应用扫描与分析结果
参与者	用户
用例描述	应用完成扫描后，用户查看分析结果
触发条件	用户查看应用详情并点击查看分析结果
前置条件	1. 系统及其所处网络环境正常运作 2. 用户已正确登录系统且拥有相关权限 3. 文件扫描任务已完成且成功
后置条件	系统返回状态码
基本流程	1. 用户点击应用详情中的查看分析结果 2. 系统显示对应应用的依赖树 3. 用户点击查看平铺依赖表 4. 系统显示对应应用的平铺依赖表
扩展流程	2a. 用户点击展开依赖树，则系统显示展开的依赖树

表3-4描述了查看组件详情用例，用户可以查看任意组件的详细信息，包含开源组件、由应用发布成的组件与上传的闭源组件。

表 3-4 查看组件详细信息用例

名称	内容描述
ID	03
用例名称	查看组件详细信息
参与者	用户查看任意组件的详细依赖信息
用例描述	用户可以查看任意一个组件的详细信息，包括该组件的依赖树和依赖列表，如果数据库中没有该组件的依赖信息，则会即时生成并存入数据库。
触发条件	用户点击查看组件详情
前置条件	1. 系统及其所处网络环境正常运作 2. 用户已正确登录系统且拥有相关权限
后置条件	系统返回状态码，如果数据库中没有依赖信息则生成并存入数据库
基本流程	1. 用户点击查看组件详情 2. 系统显示对应组件的依赖树 3. 用户点击查看平铺依赖表 4. 系统显示对应组件的平铺依赖表
扩展流程	2a. 用户点击展开依赖树，则系统显示展开的依赖树

表3-5描述了导出应用 SBOM 用例，用户可以导出 SBOM，查看应用的依赖清单与依赖关系。

表 3-5 查看导出应用 SBOM 用例

名称	内容描述
ID	04
用例名称	导出 SBOM
参与者	用户
用例描述	用户导出应用的 SBOM
触发条件	用户查看应用详情并点击导出 SBOM
前置条件	1. 系统及其所处网络环境正常运作 2. 用户已正确登录系统且拥有相关权限 3. 应用拥有子组件或应用已扫描成功
后置条件	系统返回状态码
基本流程	1. 用户查看应用详情并点击导出 SBOM 2. 系统生成对应应用的 SBOM，并将文件传输给用户下载
扩展流程	1a. 如果应用未扫描成功且不含有子组件，系统提示该应用无法导出 SBOM

表3-6描述了导出应用 Excel 用例，用户可以导出 Excel 查看应用的组件清单，包含简要和详细两种模式。

表 3-6 查看导出应用 Excel 用例

名称	内容描述
ID	05
用例名称	导出应用 Excel
参与者	用户
用例描述	用户导出应用依赖信息的 Excel，包含简要（默认）和详细两种模式
触发条件	用户查看分析结果并点击导出应用 Excel
前置条件	1. 系统及其所处网络环境正常运作 2. 用户已正确登录系统且拥有相关权限 3. 应用拥有子组件或应用已扫描成功
后置条件	系统返回状态码
基本流程	1. 用户查看分析结果并点击导出 Excel 2. 系统生成对应应用的 Excel，并将文件传输给用户下载
后置流程	1a. 用户提前框选详细导出选项，则系统会返回包含详细信息的 Excel

3.2.2 模块顺序图

Go、Python 组件依赖关系分析模块的系统顺序图如图3-2所示。

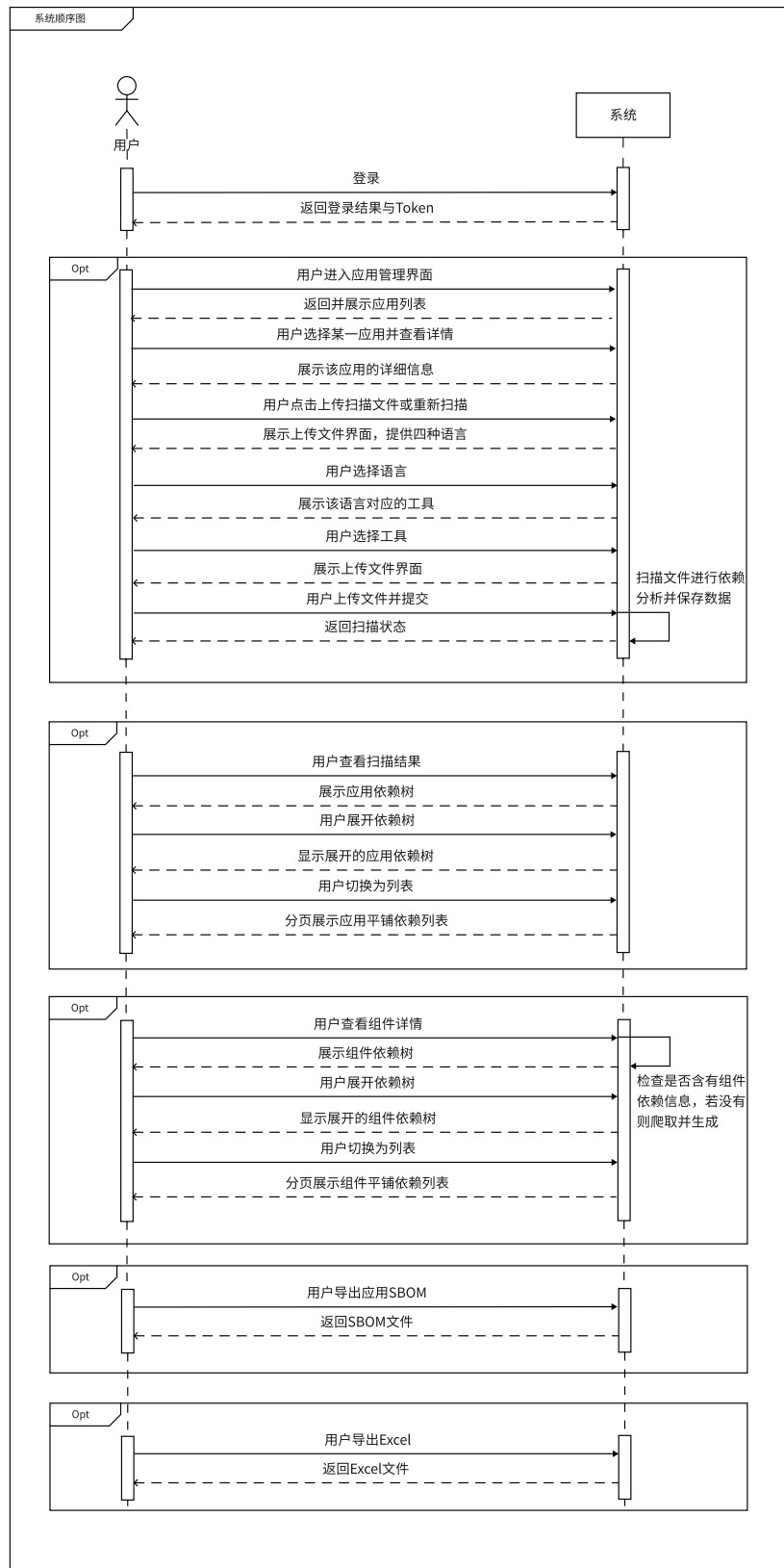


图 3-2 Go、Python 组件依赖关系分析模块系统顺序图

3.2.3 非功能需求描述

本系统的非功能需求如表3-7所示。

表 3-7 系统的非功能需求表

非功能需求要求	需求类别
网络畅通时，每个请求的响应时间不超过 5 秒钟	性能需求
系统可以同时满足 10000 个用户请求	性能需求
对 100 个知识库未知的组件的扫描分析不超过 30 分钟	性能需求
对 100 个知识库已知的组件的扫描分析不超过 5 分钟	性能需求
对于 100 万的数据级别，单个数据库查询操作不超过 1 秒钟	性能需求
对于 100 万的数据级别，单个数据库模糊查询操作不超过 10 秒钟	性能需求
只有经过验证和授权的用户才能访问系统	安全性
不同的操作需要不同的用户权限	安全性
不同部门的数据进行隔离	安全性
系统提供运行日志用于追踪历史使用情况	安全性
90% 的用户在接受 2 小时的培训后，可以熟练使用该系统	易用性
表单输入时进行提示，输入数据后进行校验	可靠性

3.2.4 约束

1. 项目后端使用 Java 11 开发
2. 组件的公共仓库的 API 未失效且能正常访问
3. 系统需要正确配置 Python 3 与 Go 1.22+ 环境
4. 必须使用 PostgreSQL 数据库

3.3 Go、Python 组件依赖关系分析模块开发总体视图

软件成分分析系统 Go、Python 组件依赖关系分析模块的开发包图如图3-3所示。

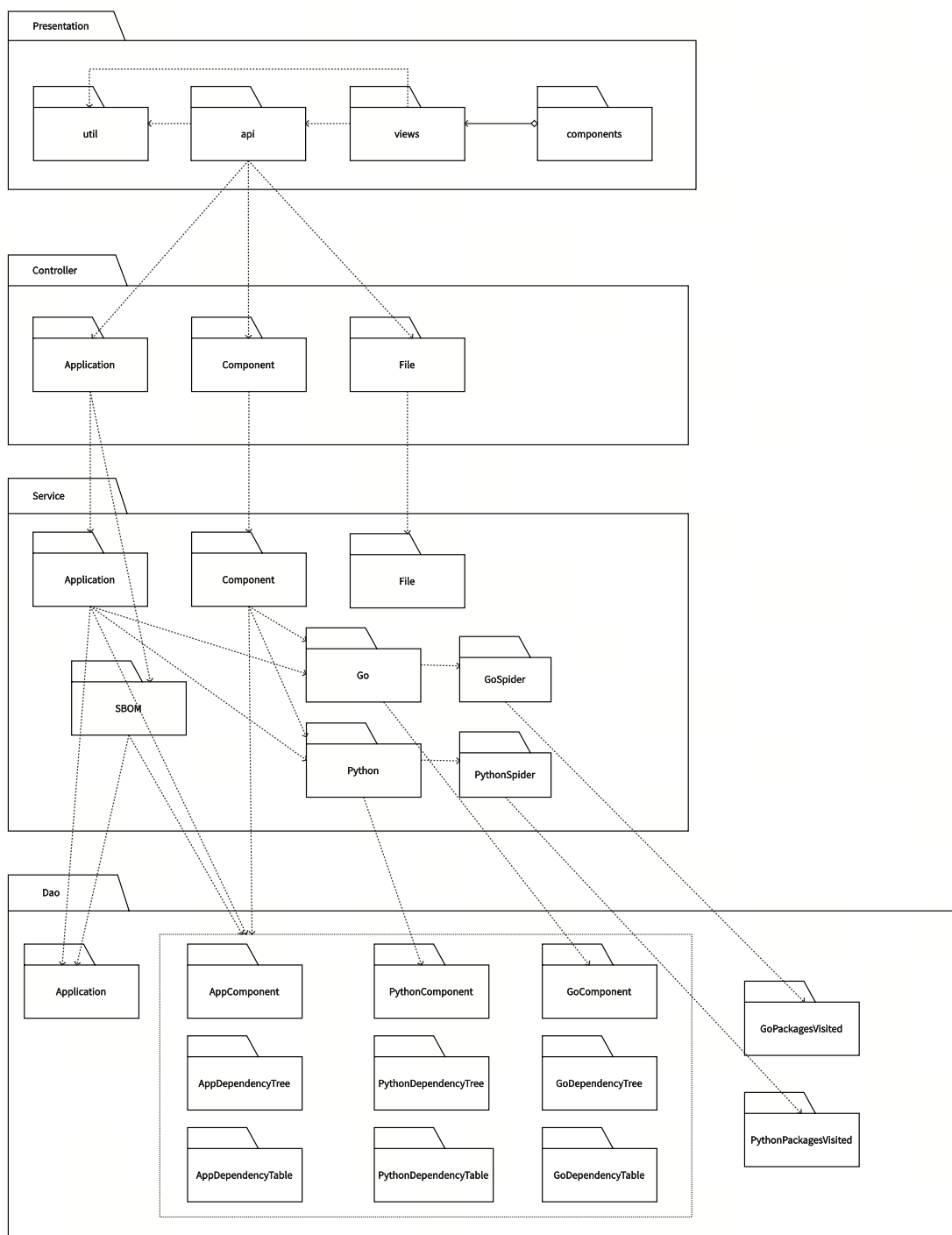


图 3-3 Go、Python 组件依赖关系分析模块开发包图

3.4 Go、Python 组件依赖关系分析模块数据库设计

3.4.1 主要实体和实体关系

本系统中 Go、Python 组件依赖关系分析模块的实体类一共有 Application（应用）、AppComponent（应用发布成的组件）、AppDependencyTree（应用依赖树）、

AppDependencyTable（应用依赖表）、GoComponent（Go 组件）、GoDependencyTree（Go 组件依赖树）、GoDependencyTable（Go 组件依赖表）、PythonComponent（Python 组件）、PythonDependencyTree（Python 组件依赖树）、PythonDependencyTable（Python 组件依赖表）。这几个实体类之间的关系如图3-4所示。

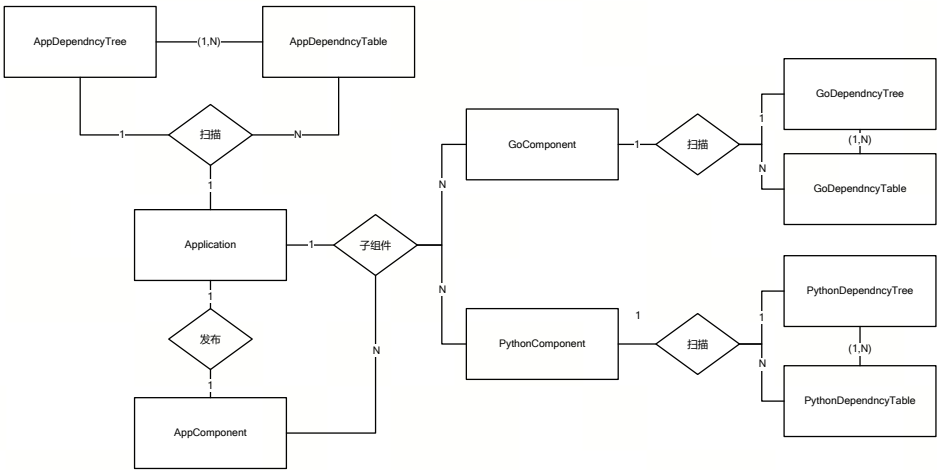


图 3-4 Go、Python 组件依赖关系分析模块实体关系图

3.4.2 表结构设计

本系统使用 PostgreSQL 作为主要数据库，Go、Python 组件依赖关系分析模块的主要实体的数据库表设计如下所示。

表3-8为 Application（应用）表，主要包含应用的各种信息。

表 3-8 plt_application 表

列名	数据类型	备注	说明
id	varchar(32)	Not Null	
name	varchar(255)	Not Null	应用名
version	varchar(255)	Not Null	应用版本号
description	text		应用描述
language	varchar[]		应用语言，可以有多个语言
type	varchar(255)	Not Null	应用类型 (开源/商用/闭源)
builder	varchar(255)		构建工具
scanner	varchar(255)		扫描对象
state	varchar(255)	Not Null	应用状态
licenses	text[]		应用许可证列表
vulnerabilities	text[]		应用漏洞列表
time	varchar(255)	Not Null	最近一次更新时间
lock	boolean	Not Null	是否上锁
release	boolean	Not Null	是否已发布
creator	varchar(32)	Not Null	创建者
child_application	text[]		子应用
child_component	jsonb		子组件

表3-9为 AppComponent（应用组件）表，主要包含由应用发布成的组件的基本信息，主要包含 id、组件名、版本号、语言、类型、描述、漏洞、许可证和多个地址等信息。

表 3-9 plt_app_component 表

列名	数据类型	备注	说明
id	varchar(32)	Not Null	
name	varchar(255)	Not Null	组件名
version	varchar(255)	Not Null	组件版本号
language	text[]	Not Null	组件语言，可以有多种语言
type	varchar(255)	Not Null	应用类型 (开源/商用/闭源)
description	text		组件描述
url	varchar(255)		主页地址
download_url	varchar(255)		下载地址
source_url	varchar(255)		源码地址
p_url	varchar(255)		包获取地址
licenses	text[]		应用许可证列表
vulnerabilities	text[]		应用漏洞列表
creator	varchar(32)		创建者
state	varchar(255)	Not Null	应用状态

表3-10为 AppDependencyTree（应用依赖树）表，主要包含应用的依赖树，其中的”tree” 字段为 jsonb 类型，是一个递归嵌套的多层结构，每一个结点下可以有多个依赖子树。

表 3-10 plt_app_dependency_tree 表

列名	数据类型	备注	说明
id	varchar(32)	Not Null	
name	varchar(255)	Not Null	组件名
version	varchar(255)	Not Null	组件版本号
tree	jsonb		依赖树

表3-11为 AppDependencyTable（应用依赖表）表，主要包含应用的依赖列表（即依赖树的平铺形式）。

表 3-11 plt_app_dependency_table 表

列名	数据类型	备注	说明
id	varchar(32)	Not Null	
name	varchar(255)	Not Null	应用名
version	varchar(255)	Not Null	应用版本号
c_name	varchar(255)	Not Null	子组件名
c_version	varchar(255)	Not Null	子组件版本号
depth	integer	Not Null	依赖层级
direct	boolean	Not Null	是否直接依赖
type	varchar(255)	Not Null	应用类型 (开源/商用/闭源)
language	varchar(255)	Not Null	应用语言
licenses	text[]	Not Null	应用许可证列表
vulnerabilities	text[]	Not Null	应用漏洞列表

表3-12为 GoComponent（Go 组件）表，主要包含知识库中 Go 组件的基本信息。

表 3-12 plt_go_component 表

列名	数据类型	备注	说明
id	varchar(32)	Not Null	
name	varchar(255)	Not Null	组件名
version	varchar(255)	Not Null	组件版本号
language	varchar(255)	Not Null	组件语言
type	varchar(255)	Not Null	组件类型 (开源/商用/闭源)
description	text		组件描述
url	varchar(255)		主页地址
download_url	varchar(255)		下载地址
source_url	varchar(255)		源码地址
p_url	varchar(255)		包获取地址
licenses	text[]		组件许可证列表
vulnerabilities	text[]		组件漏洞列表
creator	varchar(32)		创建者
state	varchar(255)	Not Null	应用状态

表3-13为 PythonComponent（Python 组件）表，主要包含知识库中 Python 组件的基本信息。

表 3-13 plt_python_component 表

列名	数据类型	备注	说明
id	varchar(32)	Not Null	
name	varchar(255)	Not Null	组件名
version	varchar(255)	Not Null	组件版本号
language	varchar(255)	Not Null	组件语言
type	varchar(255)	Not Null	组件类型 (开源/商用/闭源)
description	text		组件描述
url	varchar(255)		主页地址
download_url	varchar(255)		下载地址
source_url	varchar(255)		源码地址
p_url	varchar(255)		包获取地址
author	text		作者
author_email	text		作者邮箱
licenses	text[]		组件许可证列表
vulnerabilities	text[]		组件漏洞列表
creator	varchar(32)		创建者
state	varchar(255)	Not Null	组件状态

GoDependencyTree（Go 组件依赖树）、GoDependencyTable（Go 组件依赖表）、PythonDependencyTree（Python 组件依赖树）、PythonDependencyTable（Python 组件依赖表）所对应的数据库表 plt_go_dependency_tree、plt_go_dependency_table、plt_python_dependency_tree、plt_python_dependency_table 与 plt_app_dependency_tree、plt_app_dependency_table 两张表类似，这里不再赘述。

表3-14为 plt_visited_python_packages 表，该表维护了一个 PyPI 仓库所有 Python 包的名称和版本列表，并记录其是否被爬取过以及是否爬取成功，用作执行批量爬取任务时的目标源。plt_visited_go_packages 表与此类似，这里不再赘述。

表 3-14 plt_visited_python_packages 表

列名	数据类型	备注	说明
id	varchar(32)	Not Null	
name	varchar(255)	Not Null	组件名
version	varchar(255)	Not Null	组件版本号
visited	boolean	Not Null	是否访问过
is_success	boolean	Not Null	是否爬取成功

3.5 本章小结

本章主要介绍了软件成分分析系统的整体概述和 Go、Python 组件依赖关系分析模块的需求分析与概要设计。

首先介绍了系统的业务概要，分析了系统的目标用户、两种类型的用户，以及各自的特点、痛点和需求，并概述了系统的核心功能。

然后对 Go、Python 组件依赖关系分析模块的需求进行了分析，描述了该模块中的 5 个用例，并通过模块的顺序图描述了用户与系统模块的交互细节。同时阐述了系统的非功能需求和所要遵循的约束。

最后给出了 Go、Python 组件依赖关系分析模块的开发总体视图，描述了模块设计的内部组织形式。并给出了模块中的主要实体及其关系，以及与实体对应的数据库表设计。

第四章 Go、Python 组件依赖关系分析模块的详细设计与实现

4.1 Go、Python 组件依赖关系分析方法的设计与实现

4.1.1 Go、Python 组件依赖关系分析方法概述

Go 和 Python 组件依赖关系分析方法旨在识别给定的 Go 或 Python 项目中的依赖组件及其关系，这是 SCA 流程中的关键步骤。系统采用基于包管理器（构建）的 Build Scan 检测方法，通过识别项目中的特征文件，调用相应语言的包管理器命令获取依赖信息，并解析这些数据生成依赖树。

在不同语言的处理过程中，存在着一些系统需要应对和处理的差异。首先，用户上传项目的形式和文件格式因语言而异，例如 Go 允许上传的文件有 zip 和 go.mod 文件两种格式，Python 则有 requirements.txt、zip、tar.gz 文件三种格式。其次，各语言的包管理器在功能和特性上有所不同，例如 Go 使用 go mod 包管理器，Python 使用 pip 包管理器。最后，不同的包管理器提供的输出结果格式各不相同。

4.1.2 Go、Python 组件依赖关系分析方法详细设计

Go 组件依赖关系分析方法详细设计

Go 组件依赖关系分析流程参考图4-1。用户进行 Go 语言应用的分析扫描或上传闭源组件时，系统支持上传完整的 zip 项目或 go.mod 文件。系统会检查上传文件是否符合这两种形式之一，并确定 go.mod 文件的位置。接着，执行命令 go mod graph 获取依赖信息。如果命令执行失败，系统报告错误；否则，解析结果。

在解析 go mod graph 的结果时，通过邻接表将依赖图结构化存储。去除掉 Go 语言自身的依赖项（名称以“go@”起始的依赖）后，使用广度优先搜索（BFS）遍历结点构造依赖树，并使用 visited 数组去除重复组件。在此过程中，系统检

查依赖树中的组件在知识库中是否存在。如果不存在，系统会实时爬取并添加到知识库中。若公共仓库中找不到组件，则记录问题并跳过该组件。生成依赖树后，系统根据依赖树生成平铺依赖列表，并将依赖树和平铺依赖表存入数据库。

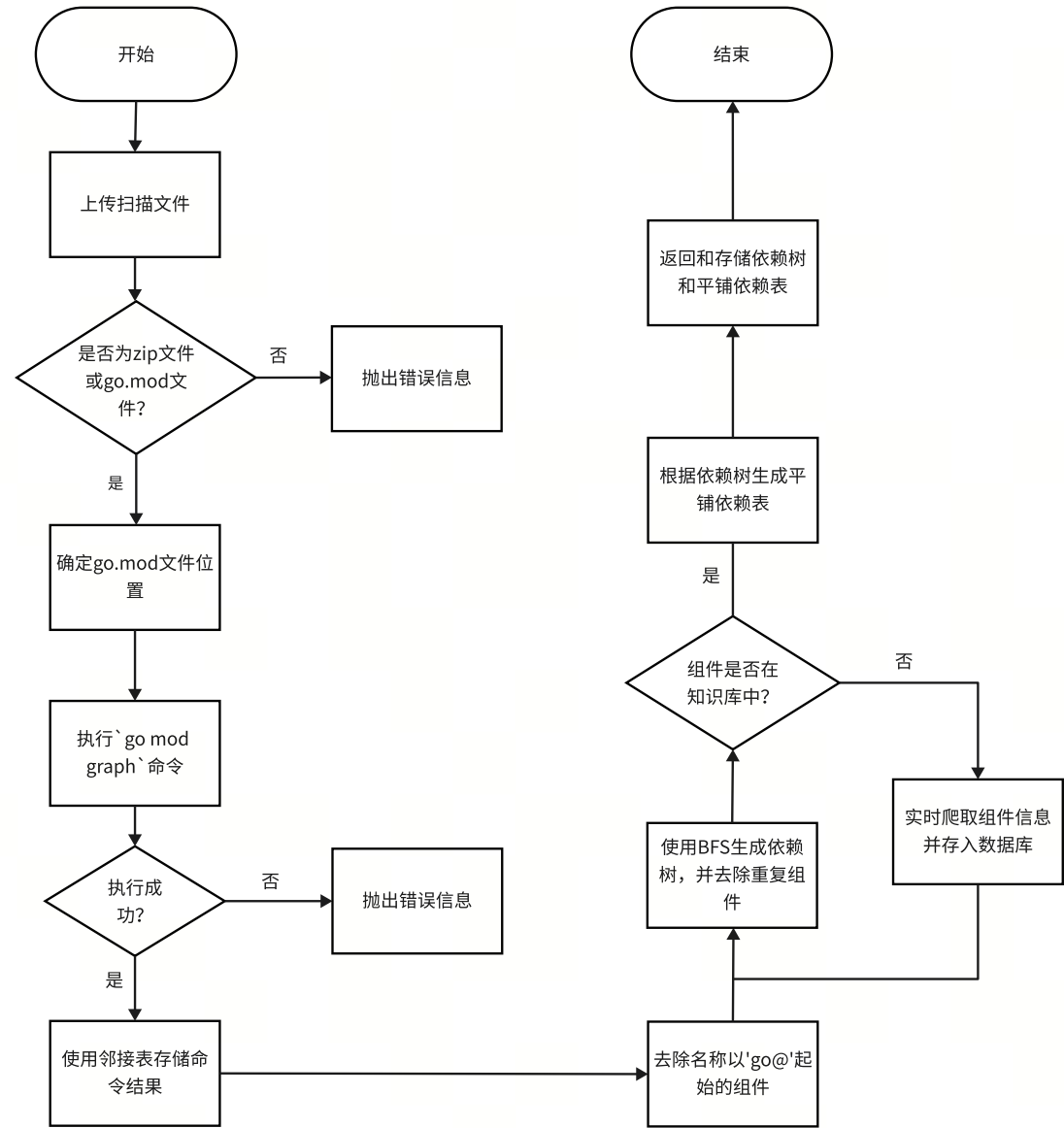


图 4-1 Go 组件依赖关系分析流程图

Python 组件依赖关系分析方法详细设计

Python 组件依赖关系分析流程参考图4-2。用户进行 Python 语言应用的分析扫描或上传闭源组件时，系统支持上传 requirements.txt、zip 或 tar.gz 文件。系统首先检查上传文件是否符合以上格式。如果不符合，则返回错误信息。

对于 zip 或 tar.gz 文件，系统先解压，并查找 setup.py 和 requirements.txt 文

件的位置。为获取项目依赖信息，首先使用 `python -m venv venv` 命令创建虚拟环境 `venv` 以隔离项目的依赖项。然后，通过 `pip install -r requirements.txt` 命令安装 `requirements.txt` 中的依赖项，并运行 `python setup.py develop` 命令安装 `setup.py` 中的依赖项（如果没有 `setup.py` 则跳过该步骤）。

接着，安装 `pipdeptree`，执行 `pipdeptree --json-tree` 命令获取 JSON 格式的依赖树信息。在解析结果之前，系统去除虚拟环境中非项目本身的依赖项，然后递归解析 JSON 格式的依赖树信息。解析完成后，通过 BFS 和 `visited` 数组去除重复依赖。生成依赖树后，系统还会生成平铺依赖列表并将其存入数据库。

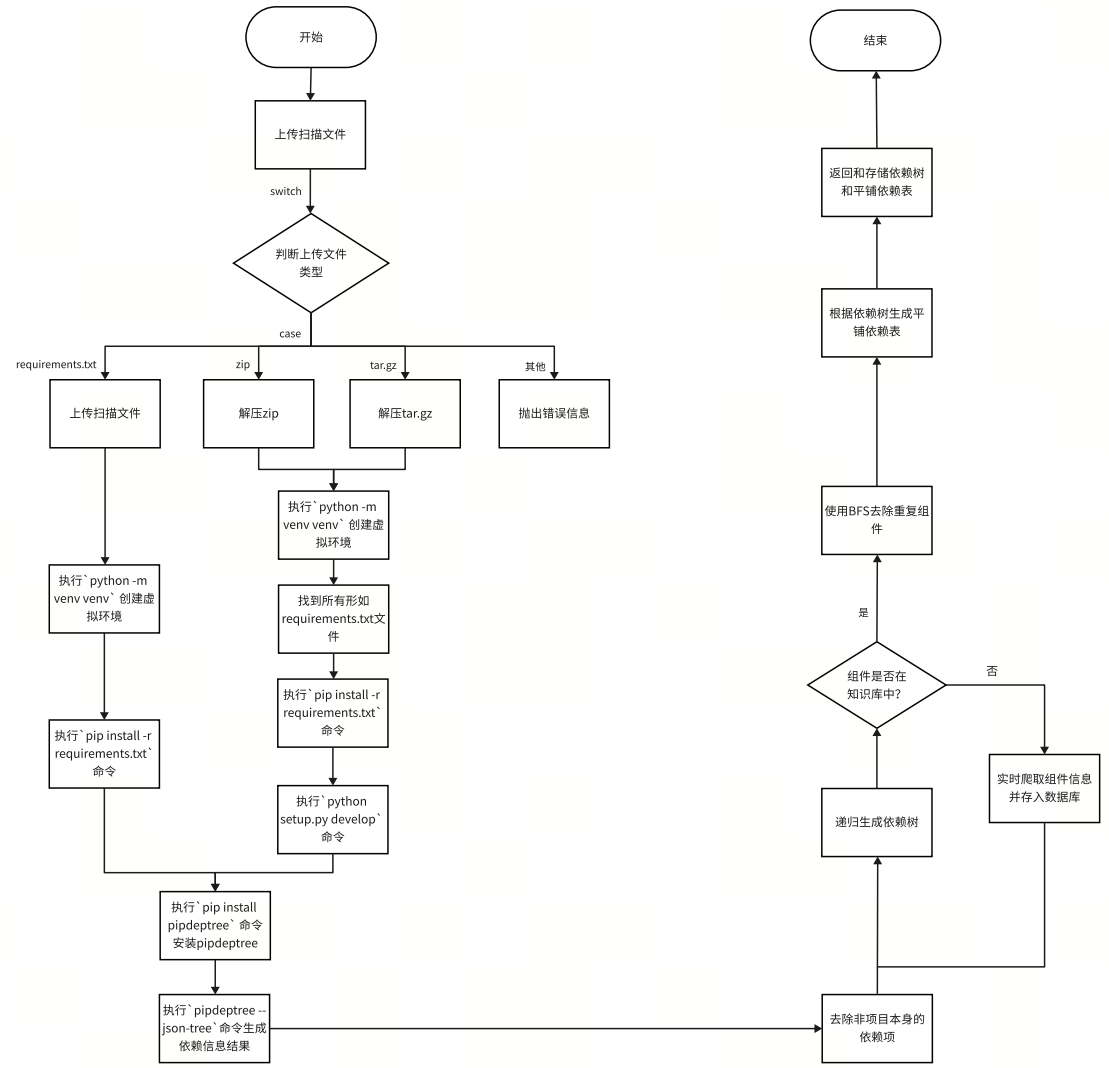


图 4-2 Python 组件依赖关系分析流程图

在执行两种语言的依赖分析过程中，都会检查依赖树中的组件在知识库中是否存在，如果不存在则实时爬取并写入知识库，爬取时如果在公共仓库中未找到该组件或发生其他错误，则系统会报告该问题并跳过该组件（但在依赖树中

仍记录该组件)。

4.1.3 Go、Python 组件依赖关系分析方法实现

Go 组件依赖关系分析方法实现

当用户上传扫描文件后，系统需要对该文件进行分析以生成项目依赖树。Go 依赖关系分析的逻辑分为三个主要步骤。

第一步为判断上传的文件类型并确定 `go.mod` 文件所在目录，系统支持处理 `zip` 文件和直接上传 `go.mod` 文件两种，对于其他文件类型则抛出错误。对于 `go.mod` 文件，工作目录为该文件所在目录，对于 `zip` 文件，工作目录为解压后的文件夹根目录。

第二步为在 `go.mod` 文件所在目录下执行 `go mod` 中的 `go mod graph` 命令并获取命令结果。具体实现上，系统使用 `ProcessBuilder` 在程序中执行命令行操作，并将直接将命令结果存储在列表中，而不写入文件或输出到控制台。若命令执行失败，系统会抛出错误。

通过执行 `go mod graph` 命令得到的依赖信息结果示例如图4-3，其中每一行记录了两个 `go` 组件，表示第一个组件依赖第二个组件，每个组件则由“@”分隔名称和版本号，多行这样的记录构成了一个依赖图。

```
hello github.com/gin-gonic/gin@v1.4.0
hello github.com/go-sql-driver/mysql@v1.4.1
hello github.com/jmoiron/sqlx@v1.2.0
hello go@1.22.1
github.com/gin-gonic/gin@v1.4.0 github.com/gin-contrib/sse@v0.0.0-20190301062529-554eab6dad3
github.com/gin-gonic/gin@v1.4.0 github.com/golang/protobuf@v1.3.1
github.com/gin-gonic/gin@v1.4.0 github.com/json-iterator/go@v1.1.6
github.com/gin-gonic/gin@v1.4.0 github.com/mattn/go-isatty@v0.0.7
github.com/gin-gonic/gin@v1.4.0 github.com/modern-go/concurrent@v0.0.0-20180306012644-bacd9c7ef1dd
github.com/gin-gonic/gin@v1.4.0 github.com/modern-go/reflect2@v1.0.1
github.com/gin-gonic/gin@v1.4.0 github.com/stretchr/testify@v1.3.0
github.com/gin-gonic/gin@v1.4.0 github.com/ugorji/go@v1.1.4
github.com/gin-gonic/gin@v1.4.0 golang.org/x/net@v0.0.0-20190503192946-f4e77d36d62c
github.com/gin-gonic/gin@v1.4.0 gopkg.in/go-playground/assert.v1@v1.2.1
github.com/gin-gonic/gin@v1.4.0 gopkg.in/go-playground/validator.v8@v8.18.2
github.com/gin-gonic/gin@v1.4.0 gopkg.in/yaml.v2@v2.2.2
github.com/jmoiron/sqlx@v1.2.0 github.com/go-sql-driver/mysql@v1.4.0
github.com/jmoiron/sqlx@v1.2.0 github.com/lib/pq@v1.0.0
github.com/jmoiron/sqlx@v1.2.0 github.com/mattn/go-sqlite3@v1.9.0
go@1.22.1 toolchain@go1.22.1
github.com/mattn/go-isatty@v0.0.7 golang.org/x/sys@v0.0.0-20190222072716-a9d3bda3a223
github.com/stretchr/testify@v1.3.0 github.com/davecgh/go-spew@v1.1.0
github.com/stretchr/testify@v1.3.0 github.com/pmezard/go-difflib@v1.0.0
github.com/stretchr/testify@v1.3.0 github.com/stretchr/objx@v0.1.0
golang.org/x/net@v0.0.0-20190503192946-f4e77d36d62c golang.org/x/crypto@v0.0.0-20190308221718-c2843e01d9a2
golang.org/x/net@v0.0.0-20190503192946-f4e77d36d62c golang.org/x/text@v0.3.0
gopkg.in/yaml.v2@v2.2.2 gopkg.in/check.v1@v0.0.0-20161208181325-20d25e280405
golang.org/x/crypto@v0.0.0-20190308221718-c2843e01d9a2 golang.org/x/sys@v0.0.0-20190215142949-d0b11bdaac8a
```

图 4-3 go mod graph 执行结果示例

第三步就是解析这种形式的结果。首先构造邻接表来存储依赖图，在排除其中 go 语言自身的依赖（名称以“go@”起始的组件）后，使用广度优先搜索（BFS）从根节点开始遍历依赖图，并用一个集合 visited 来记录访问过的组件，直到生成最后对应的项目依赖树，封装在 GoDependencyTreeDO 类中，这个过程相当于在有向图的生成树。核心代码如图4-4所示。

```
1 private GoComponentDependencyTreeDO parseTree(List<String> lines) {
2     // 读取命令行输出信息，以邻接表形式构造图
3     ...
4
5     // 通过BFS生成依赖树
6     GoComponentDependencyTreeDO tree = new GoComponentDependencyTreeDO();
7     Queue<GoComponentDependencyTreeDO> queue = new LinkedList<>();
8     queue.add(tree);
9     int depth = 0;
10    while (!queue.isEmpty()) {
11        for (int i = 0; i < queue.size(); i++) {
12            GoComponentDependencyTreeDO node = queue.poll();
13            String key = node.getName() + "@" + node.getVersion();
14            if (visited.get(key)) continue;
15            // 遍历该节点的所有边
16            for (String neighbor : graph.get(key)) {
17                if (visited.get(neighbor))
18                    continue;
19                GoComponentDependencyTreeDO child = new
20                    GoComponentDependencyTreeDO();
21                child.setDepth(depth + 1);
22                // 检查知识库中是否已有子组件信息，若无，则爬取写入知识库中
23                GoComponentDO goComponentDO = goComponentDao.find(child.
24                    getName(), child.getVersion());
25                if (null == goComponentDO) {
26                    goComponentDO = goSpiderService.crawlByNV(child.getName()
27                        , child.getVersion());
28                    goComponentDao.save(goComponentDO);
29                }
30                ...
31                node.getDependencies().add(child);
32                queue.add(child);
33            }
34            visited.put(key, true);
35        }
36        depth++;
37    }
38    return tree;
39 }
```

图 4-4 解析 go mod graph 执行结果核心代码

在生成依赖树后，系统还需要根据依赖树生成平铺的依赖列表。通过队列遍历依赖树，为每个节点生成依赖表，记录组件信息和依赖深度。核心代码如下

图4-5所示。

```
1 public List<GoDependencyTableDO> dependencyTableAnalysis(  
    GoDependencyTreeDO tree) {  
2     List<GoDependencyTableDO> tableList = new ArrayList<>();  
3     Queue<GoComponentDependencyTreeDO> queue = new LinkedList<>(tree.  
        getTree().getDependencies());  
4     while (!queue.isEmpty()) {  
5         // 将一个tree结点转化为一张table  
6         GoComponentDependencyTreeDO node = queue.poll();  
7         GoDependencyTableDO tableDO = new GoDependencyTableDO();  
8         tableDO.setName(tree.getName());  
9         ...  
10        tableList.add(tableDO);  
11        queue.addAll(node.getDependencies());  
12    }  
13    return tableList;  
14 }
```

图 4-5 由依赖树生成依赖表核心代码

Python 组件依赖关系分析方法实现

Python 组件依赖关系分析的逻辑与 Go 类似，分为三个主要步骤。

第一步为判断文件类型并确定工作目录，系统支持处理 zip、tar.gz 和 requirements.txt 三种文件类型，对于其他文件类型则抛出错误。对于 requirements.txt 文件，工作目录为该文件所在目录，对于 zip 和 tar.gz 文件，工作目录为解压后的文件夹根目录。

第二步为执行一系列命令行命令来获取项目的依赖信息，Windows 与 Linux 系统下调用这些命令的语句有一定区别，这些区别不再赘述，这里仅以 Windows 为例。需要执行的命令按顺序如下：

1. `python -m venv venv`：为了获取项目的依赖信息，需要在虚拟环境中进行隔离和管理，因此使用该命令来创建一个新的虚拟环境，接下来项目包的安装与获取依赖信息都是在该环境中完成。
2. `pip install -r requirements.txt`：一般 python 项目中都会包含 requirements.txt 文件来定义项目所需的依赖项，因此通过 pip 来安装这些依赖。在实际情况中，一些项目可能存在多个 requirements.txt 文件，并且命名上可能有些差异，因此需要先在项目中寻找这些文件。

3. `python setup.py develop`: 一般 `pytho` 项目中都会包含 `setup.py` 文件, 在这个文件中的 `install_requires` 中, 还可能要求了一些项目的依赖项, 因此需要使用 `develop` 模式执行该文件, 来安装这些依赖。
4. `pip install pipdeptree`: 通过 `pip` 安装 `pipdeptree` 工具。
5. `pipdeptree --json-tree`: 使用 `pipdeptree` 来获取虚拟环境中所有包的依赖关系信息。

在这个过程, 虽然创建了 `python` 虚拟环境但并没有激活, 这是由于是在程序中使用 `ProcessBuilder` 来执行命令行语句的, 而在一个 `ProcessBuilder` 中激活的环境并不会影响到其他 `ProcessBuilder`, 并不能像真正的命令行一样影响整个终端。因此为了确保所有操作都是在虚拟环境中执行的, 需要在执行命令时, 使用绝对或相对路径直接指定使用虚拟环境中的 `pip` 和 `python` (例如将命令改为“`D:\project\venv\Scripts\python.exe setup.py develop`”)。

第二步的核心代码如图4-6所示。

```
1 File project = new File(filePath);
2 // 创建虚拟环境
3 String[] command1 = {"python", "-m", "venv", "venv"};
4 executeCommand(command1, filePath);
5 // 先找到项目中所有形如requirements.txt的文件, 然后安装这些文件中要求的依赖
6 List<String> requirementTxtList = findRequirementFiles(project);
7 for (String requirementsTxt : requirementTxtList) {
8     String[] command2 = {project.getPath() + "\\venv\\Scripts\\python.
9         exe", "-m", "pip", "install", "-r", requirementsTxt};
10    executeCommand(command2, filePath);
11 }
12 // 安装setup.py文件中要求的依赖
13 String[] command3 = {project.getAbsolutePath() + "\\venv\\Scripts\\
14     python.exe", "setup.py", "develop"};
15 executeCommand(command3, filePath);
16 // 安装pipdeptree
17 String[] command4 = {project.getPath() + "\\venv\\Scripts\\python.exe",
18     "-m", "pip", "install", "pipdeptree"};
19 executeCommand(command4, filePath);
20 // 通过pipdeptree获取项目依赖信息
21 String[] command5 = {project.getPath() + "\\venv\\Scripts\\python.exe",
22     "-m", "pipdeptree", "--json-tree"};
23 String jsonString = executeCommand(command5, filePath);
```

图 4-6 调用 `pipdeptree --json-tree` 命令核心代码

通过执行 `pipdeptree --json-tree` 命令得到的依赖信息示例如图4-7, 其为一个

符合 json 格式的字符串。在 json 中，每个组件记录了其名称和版本，并通过“dependencies”字段递归地记录了它的依赖。

```
[
  {
    "key": "asttokens",
    "package_name": "asttokens",
    "installed_version": "2.4.1",
    "required_version": "2.4.1",
    "dependencies": [
      {
        "key": "six",
        "package_name": "six",
        "installed_version": "1.16.0",
        "required_version": ">=1.12.0",
        "dependencies": []
      }
    ]
  },
  {
    "key": "matplotlib",
    "package_name": "matplotlib",
    "installed_version": "3.7.1",
    "required_version": "3.7.1",
    "dependencies": [
      {
        "key": "contourpy",
        "package_name": "contourpy",
        "installed_version": "1.0.7",
        "required_version": ">=1.0.1",
        "dependencies": [
          {
            "key": "numpy",
            "package_name": "numpy",
            "installed_version": "1.24.3",
            "required_version": ">=1.16",
            "dependencies": []
          }
        ]
      }
    ]
  }
]
```

图 4-7 pipdeptree -json-tree 执行结果示例

第三步为解析这种形式的结果，首先去除虚拟环境中的非项目依赖（包括 pipdeptree、setuptools 和 pip 等），然后递归解析 json 格式的依赖树信息，并使用 JsonNode 构造依赖树。生成依赖树后，还需要去除依赖树中重复的组件，即当依赖树中出现相同的组件时，需要去除其中依赖层级更深的组件。核心代码如下所示。

```

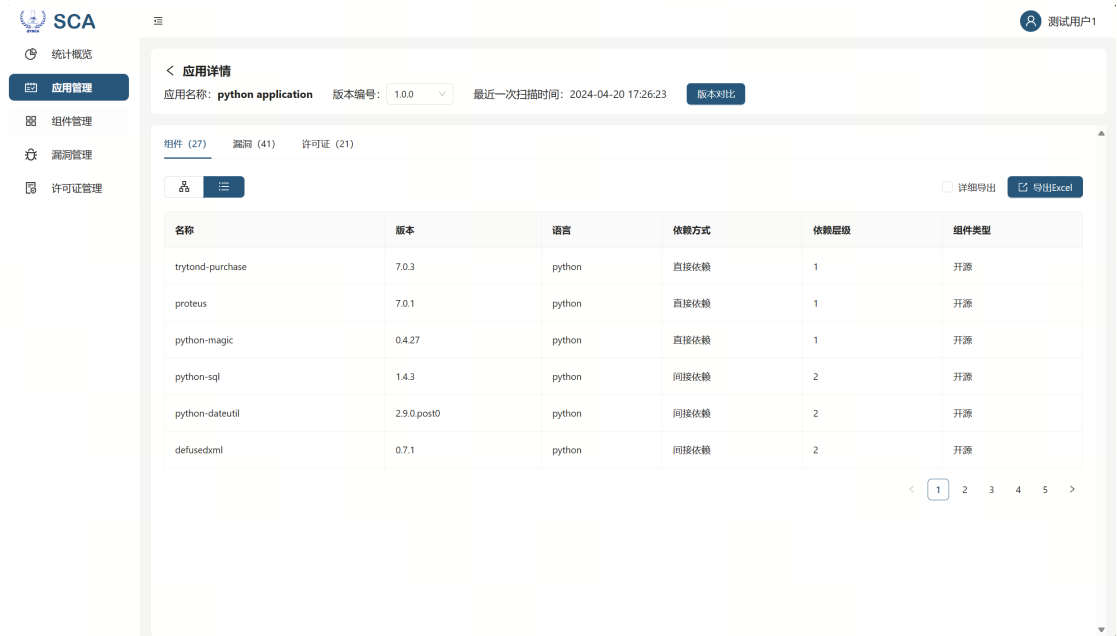
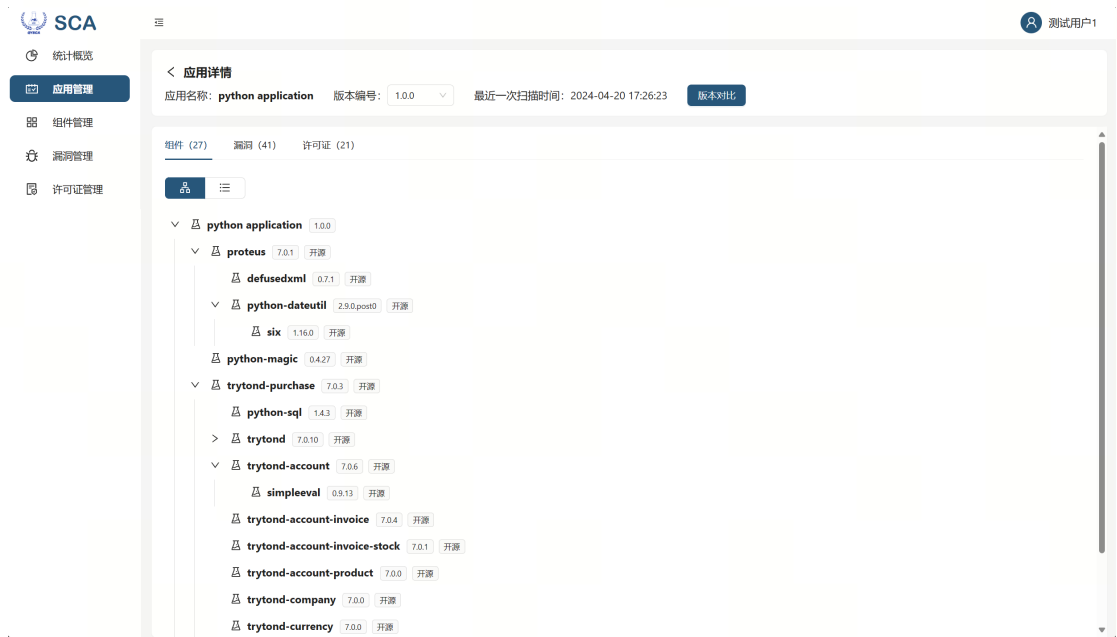
1  private PythonComponentDependencyTreeDO parseTree(JsonNode node, int
    depth) {
2      PythonComponentDependencyTreeDO tree = new
        PythonComponentDependencyTreeDO();
3      tree.setName(node.get("key").asText());
4      tree.setVersion(node.get("installed_version").asText());
5      // 从知识库中查找
6      PythonComponentDO pythonComponentDO = pythonComponentDao.
        findByNameAndVersion(tree.getName(), tree.getVersion());
7      if (pythonComponentDO == null) {
8          // 如果知识库中没有则爬取
9          pythonComponentDO = pythonSpiderService.crawlByNV(tree.getName(),
            tree.getVersion());
10         pythonComponentDao.save(pythonComponentDO);
11     }
12     tree.setDepth(depth);
13     // 递归解析依赖
14     List<PythonComponentDependencyTreeDO> dependencies = new ArrayList<>()
        ;
15     for (JsonNode child : node.get("dependencies")) {
16         dependencies.add(parseTree(child, depth + 1));
17     }
18     tree.setDependencies(dependencies);
19     return tree;
20 }

```

图 4-8 解析 pipdeptre -json-tree 执行结果核心代码

Python 通过依赖树生成平铺依赖表的实现与 Go 类似，这里不再赘述。

系统扫描和分析完成后，用户可以查看上传项目的依赖树和平铺依赖列表，其用户界面如图4-9和图4-10所示。依赖树通过可折叠的层次缩进结构显示，而依赖列表呈现为可分页的表格，详细记录了每个依赖项的依赖方式和层级。



4.2 Go、Python 组件爬取方法的设计与实现

4.2.1 Go、Python 组件爬取方法概述

在依赖解析的过程中，知识库可能缺失某些组件的信息。因此，当缺失组件信息时，通过实时爬取获取组件信息的方式来实现知识库的增量更新。此外，还

可以通过定期或触发式的批量爬取来扩展知识库。

Go 的官方公共仓库是 Go Package Index，但该仓库并不提供直接的文档访问。由于许多开源 Go 组件仓库位于 GitHub 中，可以使用 GitHub API 获取组件信息。例如，要查看 `github.com/caddyserver/caddy` 组件 v2.7.4 版本的信息，可以访问 `https://api.github.com/repos/caddyserver/caddy` 获取 json 格式的组件信息。

Python 的官方公共仓库是 PyPI，它提供了一个 API 以查询组件信息。例如，要查看 `kafka-counter` 组件 0.0.2 版本的信息，可以访问 `https://pypi.org/pypi/kafka-counter/0.0.2/json` 获取 json 格式的组件信息。

4.2.2 Go、Python 组件爬取方法详细设计

单个组件信息的爬取流程在 Go 和 Python 中基本相同，如图4-11所示。首先，根据组件的名称和版本确定需要访问的 URL，从该 URL 获取组件信息，并将其转换为对应的实体类。

在具体实现中，爬取操作通过 `httpclient` 完成，并设置请求头模拟浏览器行为，防止频繁访问导致的访问限制。Go 组件信息从 GitHub API 获取，Python 组件信息从 PyPI API 获取，二者返回的信息格式均为 json。解析使用 Jackson 库的 `JsonNode`。

而批量爬取组件信息需要先获取一个组件名称名单，爬取时按照该名单来逐一爬取并避免重复。Go 组件名单可以通过 `https://api.github.com/search/repositories?q=language:go` 获取，Python 组件名单可以通过 `https://pypi.org/pypi` 获取。

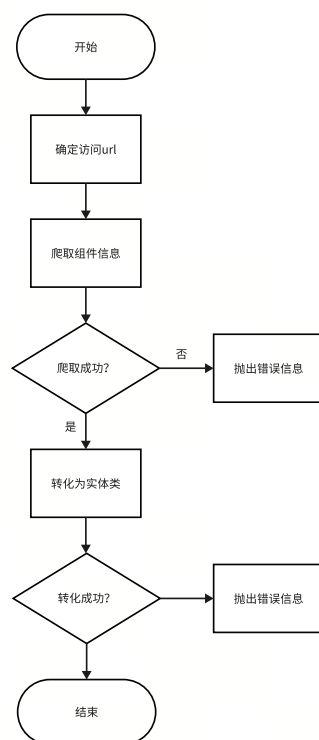


图 4-11 组件爬取流程图

4.2.3 Go、Python 组件爬取方法实现

Go 组件爬取方法实现

首先将 Github API 根地址和组件名称（去掉名称开头的“github.com/”）连接形成要访问的 url，然后创建 `httpClient` 对象，并声明访问地址。为了防止因频繁访问 api 而被 ban，还设置了请求头以模仿浏览器操作，请求头包括 `User-Agent`、`Authorization`、`Accept` 和 `X-GitHub-API-Version`。在使用 `httpClient` 发送访问请求后，会判断相应码是否为 200 OK，如果不是则抛出错误。在获取 url 中的内容之后，再使用 `JsonNode` 进行解析，获取其中的 `description`、`license` 等信息，并最终转化为实体类 `GoComponentDO`。核心代码如图4-12所示。

```

1 public GoComponentDO crawlByNV(String name, String version) {
2     // 生成要访问的url
3     String url = GO_REPO_BASE_URL + name.substring(11);
4     try {
5         // 创建HttpClient对象
6         CloseableHttpClient httpClient = HttpClients.createDefault();
7         // 声明访问地址
8         HttpGet httpGet = new HttpGet(url);
9         // 设置请求头
10        httpGet.addHeader("User-Agent", "Mozilla/5.0 (Windows NT 10.0; Win64
            ; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/38.0.101.76
            Safari/537.36");
11        httpGet.addHeader("Authorization", "Bearer
            ghp_YRoIJslH6sxxhKvQRDKiQAtOWthSOkX0w0ysM");
12        httpGet.addHeader("Accept", "application/vnd.github+json");
13        httpGet.addHeader("X-GitHub-API-Version", "2022-11-28");
14        // 发起请求
15        CloseableHttpResponse response = httpClient.execute(httpGet);
16        // 判断响应码是否为200
17        if (response.getStatusLine().getStatusCode() != 200) {
18            throw new RuntimeException();
19        }
20        // 获取内容，转化为组件实体
21        String content = EntityUtils.toString(response.getEntity(), "UTF-8"
            );
22        GoComponentDO goComponentDO = convertToGoComponentDO(content);
23        return goComponentDO;
24    } catch (Exception e) {
25        log.error("通过github api获取组件信息失败: " + name + "@" + version);
26        return null;
27    }
28 }

```

图 4-12 Go 组件爬取核心代码

对于 go 批量爬取所需要的名单，使用了一个简短的 python 代码来获取，首先访问<https://api.github.com/search/repositories?q=language:go>并获取 go 项目总数，然后再按每页一百个分页获取所有项目。同时查询时还设置了按项目的 stars 降序排序，这样可以将更受欢迎的 go 项目排在前面。相关代码如图4-13所示。

```

1  import requests
2  url = "https://api.github.com/search/repositories"
3  params = {"q": "language:go", "per_page": 100, "sort": "stars", "order": "desc"}
4  response = requests.get(url, params=params)
5  data = response.json()
6  total_count = data['total_count']
7  # 每页一百个, 获取所有go项目
8  total_pages = total_count // 100 + 1
9
10 projects = []
11 # 按页查询, 记录所有go组件名称
12 for page in range(1, total_pages + 1):
13     url = "https://api.github.com/search/repositories"
14     # 设置访问参数
15     params = {"q": "language:go", "per_page": 100, "page": {page}, "sort": "stars", "order": "desc"}
16     response = requests.get(url, params=params)
17     data = response.json()
18     for item in data['items']:
19         projects.append(item['full_name'])

```

图 4-13 获取 go 组件名单代码实现

Python 组件爬取方法实现

Python 组件爬取与 Go 基本相同, 首先将 PyPI API 根地址和组件名称、版本连接, 并在后面加上/json 形成要访问的 url, 然后同样地创建 httpclient 对象, 并声明访问地址, 设置请求头等。发送请求后判断状态码是否为 200, 然后使用 JsonNode 解析内容, 并转化为实体类 PythonComponentDO。具体代码不再赘述。

对于 python 批量爬取所需要的组件名单, 同样使用简短的 python 代码实现, 并且实现更加简单。相关代码如图4-14所示。

```

1  import xmlrpc.client
2  client = xmlrpc.client.ServerProxy('https://pypi.org/pypi')
3  package_names = client.list_packages()

```

图 4-14 获取 Python 组件名单代码实现

4.3 SBOM 与 Excel 导出功能的设计与实现

4.3.1 SBOM 与 Excel 导出功能概述

系统支持应用级别的 SBOM 和 Excel 导出。导出的 SBOM 按照 CycloneDX 标准，包含应用的所有组件信息，以及这些组件之间的依赖关系。导出的 Excel 包含简要和详尽两种模式，描述了应用所有组件的信息。

4.3.2 SBOM 与 Excel 导出功能详细设计

SBOM 导出的流程如图4-15。对于要导出的应用，有两种情况，第一种情况为该应用为底层应用，即并不含有子应用，其依赖信息是通过扫描文件生成的，此时为该应用导出一个 json 文件；第二种情况为该应用包含一个或多个子应用，并且这些子应用还可能有子应用，即该应用为一个层次结构，此时为该应用生成一个目录，为该应用的每个底层子应用生成一个 json 文件，用目录嵌套表示层级结构，最终将目录打包导出成一个 zip 文件。导出应用 SBOM 时，先准备需要的数据，包括应用信息、应用的组件信息以及组件之间的依赖关系信息，然后将这些信息封装入一个 SbomDTO 中，再将该 SbomDTO 转化为 json 形式写入文件，返回给 HttpServletResponse。

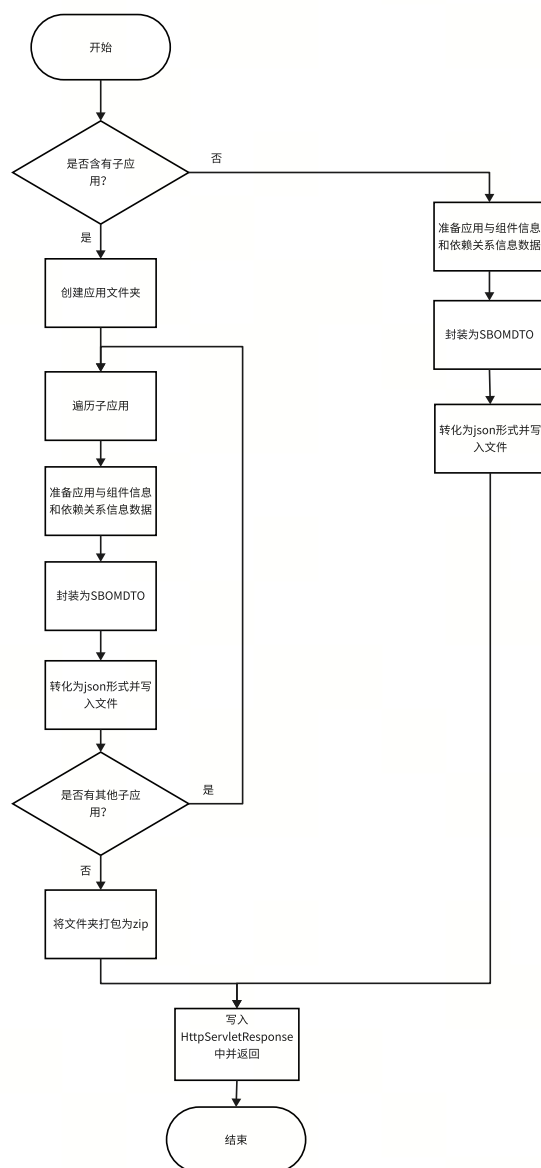


图 4-15 导出 SBOM 流程图

Excel 导出的流程如图4-16。系统查询应用的平铺依赖表，然后按每个组件一行导出成 Excel。对于简略模式，只包含组件的名称、版本、语言、依赖方式、依赖层级和类型六个字段；对于详尽模式，还包含组件描述、各种地址、开发者、许可证等信息。

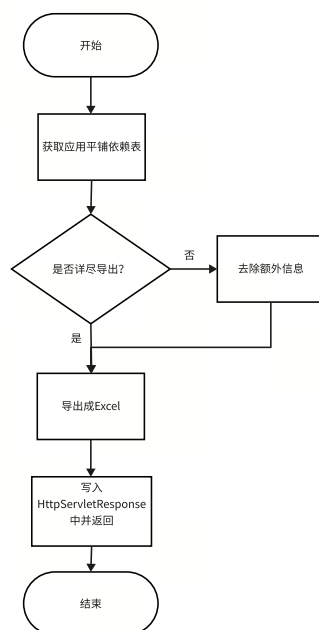


图 4-16 导出 Excel 流程图

4.3.3 SBOM 与 Excel 导出功能实现

SBOM 导出功能实现

在导出 SBOM 时，系统首先判断应用是否包含子应用。如果没有子应用，则将其视为底层应用。

对于底层应用的 SBOM 导出，流程如下：

1. 获取应用信息：根据应用名称和版本查询数据库，获取应用及相关信息。
2. 获取组件列表：查询应用的平铺依赖表，获得应用的所有组件列表。
3. 获取依赖关系：查询应用的依赖树，并使用队列逐层解析，获取组件间的依赖关系。
4. 封装数据：将上述信息封装到 SbomDTO 中。
5. 写入 json 文件：使用 Jackson 的 ObjectMapper 将 SbomDTO 转换为 json 字符串，并将其写入文件。
6. 写入 HttpServletResponse 请求并返回。

在实现过程中，系统需要考虑应用和组件的语言差异。不同语言导出的 SBOM 内容格式可能不同，因此 SbomDTO 包含一个抽象类字段 SbomComponentDTO，每种语言的 SBOM 继承自该抽象类，并相应调整字段。

核心代码如图4-17所示。

```
1 // 查询应用的平铺依赖表，获取其所有组件的列表
2 List<AppDependencyTableDO> appDependencyTableDOS =
    appDependencyTableDao.findAllByNameAndVersion(applicationDO.
        getName(), applicationDO.getVersion());
3 // 将每个组件封装为SbomComponentDTO
4 List<SbomComponentDTO> sbomComponentDTOList = new ArrayList<>();
5 for (AppDependencyTableDO dependencyTable : appDependencyTableDOS) {
6     sbomComponentDTOList.add(getComponentDTO(dependencyTable.getCName
        (), dependencyTable.getCVersion(), dependencyTable.getDirect(),
        dependencyTable.getLanguage()));
7 }
8
9 // 查询应用的依赖树，获取这些组件的依赖关系
10 AppComponentDependencyTreeDO root = appDependencyTreeDao.
    findByNameAndVersion(applicationDO.getName(), applicationDO.
        getVersion()).getTree();
11 List<SbomDependencyDTO> sbomDependencyDTOList= getSbomDependencyList(
    root);
12
13 // 封装为SbomDTO
14 SbomDTO sbomDTO = new SbomDTO(applicationDO, sbomComponentDTOList,
    sbomDependencyDTOList);
15 // 生成json文件
16 File file = makeSbomFile(tempDir, sbomDTO);
17 // 写入HttpServletResponse请求，导出
18 export(response, file);
19 }
```

图 4-17 底层应用导出 SBOM 核心代码

对于非底层应用，系统首先创建一个文件夹，然后递归遍历所有子应用。如果子应用是底层应用，则在文件夹中为该子应用按上述逻辑生成一个 JSON 文件。如果子应用不是底层应用，则在文件夹中创建一个子文件夹，并继续递归遍历该子应用。最终，系统会生成一个分层的目录结构，并将其压缩为 ZIP 文件返回。核心代码如图 4-18所示。


```

1 private void exportSBOM(String appId, File dir) {
2     ApplicationDO applicationDO = applicationDao.findOneById(appId);
3     if (applicationDO.getChildApplication().length==0) {
4         // 如果应用没有子应用，那么生成一个json文件
5         AppComponentDO appComponentDO = appComponentDao.
            findByNameAndVersion(applicationDO.getName(), applicationDO.
                getVersion());
6         SbomDTO sbomDTO = getSbomDTO("app", appComponentDO.getId());
7         makeSbomFile(dir, sbomDTO);
8     } else {
9         // 如果应用还有子应用，那么先生成一个文件夹
10        File sonDir = new File(dir, applicationDO.getName().replaceAll(":",
            "-"));
11        // 为每个子组件生成一个json
12        for (Map.Entry<String, List<String>> entry : applicationDO.
            getChildComponent().entrySet())
13            for (String childComponentId : entry.getValue()) {
14                SbomDTO sbomDTO = getSbomDTO(entry.getKey(),
                    childComponentId);
15                makeSbomFile(sonDir, sbomDTO);
16            }
17        // 继续递归导出子应用
18        for (String childAppId : applicationDO.getChildApplication())
19            exportSBOM(childAppId, sonDir);
20    }
21 }

```

图 4-18 非底层应用导出 SBOM 核心代码

用户导出后可以下载 SBOM，对于底层应用导出的 SBOM，其结果示例如图4-19。在示例中，包含了应用的名称、版本等信息以及 SBOM 生成的时间，并在“components”字段中描述了该应用所有组件的信息，“dependencies”字段中描述了这些组件之间的依赖关系。

对于包含子应用的应用导出的 SBOM，其目录结构示例如图4-20。在示例中，顶层应用下包含 backend、frontend、scripts 三个子应用，这三个子应用又分别包含一个或多个底层应用，每个底层应用会生成一个 json 文件存储 SBOM。

```

{
  "name": "python application",
  "version": "1.0.0",
  "timestamp": "2024-05-05 09:36:59 UTC",
  "components": [
    {
      "origin": "opensource",
      "name": "proteus",
      "version": "7.0.1",
      "language": "python",
      "description": "=====\nTryton Scripting Client\n=====\n\nA",
      "licenses": [],
      "hashes": [],
      "externalReferences": [ ...
    ],
      "purl": "pkg:pypi/proteus@7.0.1"
    },
    {
      "origin": "opensource",
      "name": "python-magic",
      "version": "0.4.27",
      "language": "python",
      "description": "",
      "licenses": [],
      "hashes": [],
      "externalReferences": [ ...
    ],
      "purl": "pkg:pypi/python-magic@0.4.27"
    }
  ],
  "dependencies": [
    {
      "ref": "pkg:pypi/proteus@7.0.1",
      "dependsOn": [
        "pkg:pypi/defusedxml@0.7.1",
        "pkg:pypi/python-dateutil@2.9.0.post0"
      ]
    },
    {
      "ref": "pkg:pypi/python-magic@0.4.27",
      "dependsOn": []
    }
  ]
}

```

图 4-19 底层应用 SBOM 导出结果（部分）示例

```

▼ SBOM
  ▼ backend
    {} backend_clickhouse-1.0.0-SBOM.json
    {} com.nju.edu_qysca-1.0.0-SBOM.json
    {} go application-2.0.0-SBOM.json
    {} gradle application-1.0.0-SBOM.json
  ▼ frontend
    {} frontend-1.0.0-SBOM.json
  ▼ scripts
    {} python application-1.0.0-SBOM.json

```

图 4-20 非底层应用 SBOM 导出结果示例

Excel 导出功能实现

导出 Excel 的实现逻辑比较简单，首先查询要导出的应用的平铺依赖表，然后根据是简略还是详尽模式选择需要的字段，最后导出成 Excel 即可。以简略模式为例，核心代码如图4-21所示。

```
1      List<TableExcelBriefDTO> resList = appDependencyTableDao.  
        findTableListByNameAndVersion(applicationSearchDTO.getName(),  
        applicationSearchDTO.getVersion());  
2      String fileName = applicationSearchDTO.getName().replace(":", "-") + "  
        -" + applicationSearchDTO.getVersion() + "-dependencyTable-brief";  
3      try {  
4          ExcelUtils.export(response, fileName, resList, TableExcelBriefDTO.  
              class);  
5      } catch (Exception e) {  
6          throw new PlatformException(500, "导出Excel失败");  
7      }
```

图 4-21 导出 Excel 核心代码

导出的 Excel（简略）结果示例如图4-22。其中每一行表示一个应用依赖的一个组件，里面记录了组件的名称、版本、语言、类型、依赖方式等信息、依赖层级。在详尽模式中，还会记录组件的描述、各种地址、许可证、作者等更多信息。

1	名称	版本	语言	依赖方式	依赖层级	类型
2	proteus	7.0.1	python	直接依赖	1	opensource
3	python-magic	0.4.27	python	直接依赖	1	opensource
4	trytond-purchase	7.0.3	python	直接依赖	1	opensource
5	defusedxml	0.7.1	python	间接依赖	2	opensource
6	python-dateutil	2.9.0.post0	python	间接依赖	2	opensource
7	python-sql	1.4.3	python	间接依赖	2	opensource
8	trytond	7.0.10	python	间接依赖	2	opensource
9	trytond-account	7.0.6	python	间接依赖	2	opensource
10	trytond-account-invoice	7.0.4	python	间接依赖	2	opensource
11	trytond-account-invoice	7.0.1	python	间接依赖	2	opensource
12	trytond-account-product	7.0.0	python	间接依赖	2	opensource
13	trytond-company	7.0.0	python	间接依赖	2	opensource
14	trytond-currency	7.0.0	python	间接依赖	2	opensource
15	trytond-party	7.0.2	python	间接依赖	2	opensource
16	trytond-product	7.0.1	python	间接依赖	2	opensource
17	trytond-stock	7.0.4	python	间接依赖	2	opensource
18	six	1.16.0	python	间接依赖	3	opensource
19	genshi	0.7.7	python	间接依赖	3	opensource
20	lxml	5.2.1	python	间接依赖	3	opensource
21	passlib	1.7.4	python	间接依赖	3	opensource
22	polib	1.2.0	python	间接依赖	3	opensource
23	relatorio	0.10.1	python	间接依赖	3	opensource
24	werkzeug	3.0.2	python	间接依赖	3	opensource
25	simpleeval	0.9.13	python	间接依赖	3	opensource
26	python-stdnum	1.20	python	间接依赖	3	opensource
27	trytond-country	7.0.0	python	间接依赖	3	opensource
28	markupsafe	2.1.5	python	间接依赖	4	opensource

图 4-22 导出 Excel（简略）结果示例

4.4 本章小结

本章详细介绍了系统中 Go 和 Python 组件依赖关系分析模块的设计和实现。核心思路是通过基于包管理器的构建扫描方法，获取项目的依赖和依赖关系信息。针对 Go 和 Python 两种语言及其包管理器的特点，系统采用了相应的定制化处理。

此外，本章还介绍了由依赖关系分析扩展出的 Go 和 Python 组件爬取方法的设计和实现。基本思路是使用 httpclient，通过调用相应语言的公共仓库 API 获取组件信息，并将其转换为数据库实体类存储在知识库中。

最后，本章探讨了导出 SBOM 和 Excel 的两个 SCA 关键功能的设计与实现，特别是多层次应用场景下 SBOM 的导出设计。

在详细阐述设计和实现时，本文通过流程图直观呈现了各模块的整体操作流程，并结合相关代码具体描述了核心解决方案。同时，展示了与之相关的前端界面，直观体现了系统中相关功能的呈现效果。

第五章 Go、Python 组件依赖关系分析模块测试

5.1 测试环境

本章的测试环境如表5-1所示。

表 5-1 测试环境表

设备与环境	描述
服务器	2 核 CPU，4GB 内存
操作系统	CentOS 8
JDK	Java 11
Python	3.9
Go	1.22.1
Node.js	16
PostgreSQL	16.0
Redis	7.0

5.2 功能测试

本系统对 Go、Python 组件依赖关系分析模块进行了全面的功能测试，测试设计和结果如表5-2所示。

表 5-2 功能测试表

用 例 编号	测试用例描述	操作过程及数据	预计结果	测 试 结果
D001	应用分析	用户查看一个空应用的详情，然后点击上传扫描文件	显示上传扫描文件界面	通过

表 5-2 功能测试表（续）

用例编号	测试用例描述	操作过程及数据	预计结果	测试结果
D002	上传闭源组件	用户添加组件，填写组件名称、版本号、描述、类型，然后上传扫描文件	显示上传扫描文件界面	通过
D003	上传扫描文件	用户选择语言、工具并上传文件，点击“确定”	提示上传成功并显示正在扫描	通过
D004	上传文件与选择的语言工具不符	用户选择语言、工具，然后上传了不符合要求的文件类型，点击“确定”	提示文件类型错误	通过
D005	上传的文件的信息与应用或组件信息不符	用户上传的扫描文件与应用或组件本身的信息不符	提示对应信息不符	通过
D006	上传可扫描成功的文件	上传扫描文件完成后，系统进行分析	系统应能正确分析并生成和存储分析结果	通过
D007	上传无法扫描成功的文件	上传扫描文件完成后，系统进行分析	显示扫描失败	通过
D008	扫描文件中存在无法识别的组件	扫描文件中存在无法识别的组件	扫描分析过程跳过该组件正常进行	通过
D009	重新扫描	扫描成功或失败后，进行重新扫描	重复上传扫描文件流程	通过
D010	查看分析结果	选择应用或组件，查看详情，然后查看分析结果	显示对应应用或组件的依赖树	通过
D011	查看平铺依赖表	查看依赖树界面，切换为查看依赖表	分页显示对应应用或组件的平铺依赖列表	通过
D012	导出底层应用 SBOM	查看一个底层应用的详情，点击导出 SBOM	导出一个 json 文件，内容为对应 SBOM	通过

表 5-2 功能测试表（续）

用例编号	测试用例描述	操作过程及数据	预计结果	测试结果
D013	导出多层次结构应用 SBOM	查看一个多层次结构应用的详情，点击导出 SBOM	导出一个 zip 文件，里面包含嵌套目录结构和多个 SBOM json 文件	通过
D014	导出空应用 SBOM	查看一个空应用的详情（没有进行依赖扫描），点击导出 SBOM	提示无法导出空应用 SBOM	通过
D015	导出应用 Excel（简略）	查看应用依赖表，点击导出 Excel	导出一个包含简略依赖信息的 Excel	通过
D016	导出应用 Excel（详细）	查看应用依赖表，先框选详细导出，再点击导出 Excel	导出一个包含详细依赖信息的 Excel	通过

5.3 本章小结

本章对 Go、Python 组件依赖关系分析模块进行有效的测试，基本保证了该模块功能具备较好的稳定性和可用性。

第六章 总结与展望

6.1 总结

随着软件开发的不断进步和复杂性的增加，现代应用程序往往依赖于许多第三方库和组件，这些组件经常由不同来源的开发者编写。为了保证软件的质量、安全性和可维护性，了解项目中所有组件的来源、版本、许可证、漏洞和依赖关系变得至关重要。基于此行业背景，基于这一背景，本文提出了具有应用管理功能的面向开源、商业或闭源多种不同类型组件的轻量级的软件成分分析系统，并重点探讨了其中 Go 和 Python 组件依赖关系分析模块的设计与实现。

首先，本文深入探讨了系统的背景，包括系统的产生原因、必要性和意义，同时简要介绍了国内外 SCA 发展情况以及一些 SCA 产品的特点。接着，详细介绍了系统所采用的主要技术栈，包括 Vue 与 Spring Boot 构建的前后端框架、Redis 内存数据库和 PostgreSQL 主数据库。还讨论了软件成分分析领域的三种主流检测技术，并详述了系统最终选择基于包管理器（构建）中的 Build Scan 作为主要检测方案的原因。此外，本文还介绍了 Python 与 Go 两种语言的依赖管理方式和公共仓库，以及软件物料清单（SBOM）的概念。

接下来，本文从用户需求和核心功能的角度对系统进行了整体概述，并对 Go 和 Python 组件依赖关系分析模块进行了深入的需求分析。通过用例图和详细的用例描述，阐明了模块的目标和预期行为。同时，本文提供了反映用户行为顺序的系统顺序图，展示了系统在满足用户需求的过程中各步骤的交互和依赖关系。详细描述了系统的非功能需求和约束条件，这些要求是确保系统性能、可扩展性和可靠性的重要依据。在需求分析的基础上，本文探讨了模块的开发总体视图、主要实体和实体关系以及数据表的设计。这些设计决定了系统数据的存储和组织方式，有助于确保系统在处理的效率和一致性。

随后，本文详细介绍了 Go 和 Python 组件依赖关系分析模块的设计与实现，并使用流程图和用户界面图帮助读者深入理解系统的行为逻辑和模块的核心功

能。同时，本文解释了模块功能中的相关核心代码，进一步阐明了其设计理念和实现细节。

最后，本文在系统的测试环节进行了全面、详尽且严谨的测试工作。通过测试，确保了系统的可靠性和稳定性，验证了系统的各项功能是否满足预期的需求。

总体而言，本文的研究结果为开发团队提供了一种切实可行的轻量级的软件成分分析工具，帮助他们在现代软件开发过程中应对复杂的供应链挑战，并保障项目的安全性和合规性。

6.2 下一步工作

由于软件成分分析系统的复杂性以及开发时间和资源的限制，本系统仍有许多改进和扩展的空间。在下一步的工作中，我们将重点关注以下几个方面：

第一，本系统最终选择了基于包管理器（构建）的 **Build Scan** 作为检测技术，这一选择在现阶段取得了良好的效果，但并不意味着其他检测技术没有其优势。在未来的发展中，系统可以扩展使用其他检测技术，例如基于包管理器（构建）的 **Pre-Build Scan** 检测、基于源码相似度匹配的检测和基于二进制的开源软件识别技术。这些检测技术可以根据不同的需求场景灵活选择，以提高系统的适应性和全面性。

第二，本系统在设计组件爬取时，仅考虑了各语言的官方公共仓库，这可能导致某些开源组件信息的缺失。为解决这个问题，未来系统可以增加对多个公共仓库的支持，并允许用户手动添加新的仓库地址。通过在爬取时尝试从多个仓库获取组件信息，系统能够更全面地覆盖开源组件。

第三，本系统设计了用户权限管理、应用管理和组件管理功能，但与 **DevOps** 和自动化的融合仍不够完善。未来的发展方向应侧重于与 **DevOps** 流程的深度集成，借助自动化工具增强系统的灵活性和效率。通过实现持续集成、持续交付的支持，系统可以更好地融入现代开发流程，提升项目的效率和质量。

第四，随着开源和商业软件的不断发展，系统可以探索对更多编程语言的支持，包括 **C**、**C++**、**Ruby** 等。这将增强系统的适用性，满足更多开发者的需求。同时，系统应关注数据隐私和安全性，确保在分析和爬取数据时，遵循相关法规

和最佳实践。

第五，系统未来可以考虑添加更多报告和可视化功能。例如，定制化的依赖关系图表、可视化仪表盘、更多格式的 SBOM 和风险评估报告等。这些功能将帮助开发者和安全团队更直观地理解项目的状态，提高决策的准确性和效率。

参考文献

- [1] WORTHINGTON J. The Forrester Wave™: Software Composition Analysis, Q2 2023[R]. Forrester, 2023.
- [2] MCBEE M. State of Software Security: Open Source Edition[R]. Veracode, 2020.
- [3] 2024 Open Source Security and Risk Analysis Report[R]. Synopsys, 2024.
- [4] HASSIJA V, CHAMOLA V, GUPTA V, et al. A Survey on Supply Chain Security: Application Areas, Security Threats, and Solution Architectures[J]. IEEE Internet of Things Journal, 2021, 8(8): 6222-6246. DOI: 10.1109/JIOT.2020.3025775.
- [5] FOO D, YEO J, XIAO H, et al. The Dynamics of Software Composition Analysis[Z]. 2019. arXiv: 1909.00973 [cs.SE].
- [6] GOKKAYA B, KARAFILI E, ANIELLO L, et al. Global supply chains security: a comparative analysis of emerging threats and traceability solutions[J/OL]. Benchmarking: An International Journal, 2024, ahead-of-print(ahead-of-print). <https://doi.org/10.1108/BIJ-08-2023-0535>. DOI: 10.1108/BIJ-08-2023-0535.
- [7] BELGUIDOUM M, DAGNAT F. Dependency Management in Software Component Deployment[J/OL]. Electronic Notes in Theoretical Computer Science, 2007, 182: 17-32. <https://www.sciencedirect.com/science/article/pii/S1571066107003830>. DOI: <https://doi.org/10.1016/j.entcs.2006.09.029>.
- [8] PASHCHENKO I, VU D L, MASSACCI F. A Qualitative Study of Dependency Management and Its Security Implications[C/OL]//CCS '20. Virtual Event, USA: Association for Computing Machinery, 2020: 1513-1531. <https://doi.org/10.1145/3372297.3417232>. DOI: 10.1145/3372297.3417232.

- [9] MOODUTO A, RIJANTO E, PAMUJI G C. Optimization of Software Development Automation via CICD, Dependency Track, and AWS CodePipeline Integration[C]//2023 International Conference on Informatics Engineering, Science & Technology (INCITEST). 2023: 1-7. DOI: 10.1109/INCITEST59455.2023.10397047.
- [10] MACKEY T. Building open source security into agile application builds[J/OL]. Network Security, 2018, 2018(4): 5-8. <https://www.sciencedirect.com/science/article/pii/S1353485818300321>. DOI: [https://doi.org/10.1016/S1353-4858\(18\)30032-1](https://doi.org/10.1016/S1353-4858(18)30032-1).
- [11] LEE G Q. SCA vs. SCA : a comparison of SCA tools in the market[J/OL]. Final Year Project (FYP), 2021. <https://hdl.handle.net/10356/148846>.
- [12] ALJOHANI M A, ALQAHTANI S S. A unified framework for automating software security analysis in DevSecOps[C]//2023 International Conference on Smart Computing and Application (ICSCA). 2023: 1-6.
- [13] NELSON B. Getting to know Vue. js[J]. Getting to Know Vue. js, 2018.
- [14] FILIPOVA O. Learning vue. js 2[M]. Packt Publishing Ltd, 2016: 25-26.
- [15] WALLS C. Spring in action[M]. Simon, 2022: 36-39.
- [16] SMITH G. PostgreSQL 9.0 High Performance[M]. Packt Publishing, 2010: 124-125.
- [17] DRAKE J D, WORSLEY J C. Practical PostgreSQL[M]. ” O’Reilly Media, Inc.”, 2002: 85-87.
- [18] DECAN A, MENS T, CLAES M. On the topology of package dependency networks: a comparison of three programming language ecosystems[C/OL]//ECSAW ’16: Proceedings of the 10th European Conference on Software Architecture Workshops. Copenhagen, Denmark: Association for Computing Machinery, 2016. <https://doi.org/10.1145/2993412.3003382>. DOI: 10.1145/2993412.3003382.

- [19] PASHCHENKO I, VU D L, MASSACCI F. A Qualitative Study of Dependency Management and Its Security Implications[C/OL]//CCS '20: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security. Virtual Event, USA: Association for Computing Machinery, 2020: 1513-1531. <https://doi.org/10.1145/3372297.3417232>. DOI: 10.1145/3372297.3417232.
- [20] MARCELLI A, GRAZIANO M, UGARTE-PEDRERO X, et al. How Machine Learning Is Solving the Binary Function Similarity Problem[C/OL]//31st USENIX Security Symposium (USENIX Security 22). Boston, MA: USENIX Association, 2022: 2099-2116. <https://www.usenix.org/conference/usenixsecurity22/presentation/marcelli>.
- [21] HEMEL A, KALLEBERG K T, VERMAAS R, et al. Finding software license violations through binary code clone detection[C/OL]//MSR '11: Proceedings of the 8th Working Conference on Mining Software Repositories. Waikiki, Honolulu, HI, USA: Association for Computing Machinery, 2011: 63-72. <https://doi.org/10.1145/1985441.1985453>. DOI: 10.1145/1985441.1985453.
- [22] CAO Y, CHEN L, MA W, et al. Towards Better Dependency Management: A First Look at Dependency Smells in Python Projects[J]. IEEE Transactions on Software Engineering, 2023, 49(4): 1741-1765. DOI: 10.1109/TSE.2022.3191353.
- [23] BOMMARITO E, BOMMARITO M. An Empirical Analysis of the Python Package Index (PyPI)[Z]. 2019. arXiv: 1907.11073 [cs.SE].
- [24] Bin UZAYR S. GoLang: The Ultimate Guide[M/OL]. 1st ed. CRC Press, 2022: 166-168. <https://doi.org/10.1201/9781003309055>. DOI: 10.1201/9781003309055.
- [25] BI T, XIA B, XING Z, et al. On the Way to SBOMs: Investigating Design Issues and Solutions in Practice[J/OL]. ACM Trans. Softw. Eng. Methodol., 2024. <https://doi.org/10.1145/3654442>. DOI: 10.1145/3654442.

致 谢

首先，我要特别感谢我的导师刘峰老师。刘峰老师为我们提供了这项富有意义且充满挑战的课题，并在系统开发和运维上提供了充分的资源支持。此外，刘峰老师为整个研究过程提供了许多宝贵的专业指导和耐心帮助。他的指导让我保持了正确的研究方向，并激发了我的学术思维。

其次，我要感谢课题组的袁家乐、田姚和黄宝俊三位同学。在与他们合作的过程中，他们的支持和合作对我的研究进展起到了至关重要的作用。他们的经验和见解帮助我克服了许多挑战，提高了我的认知和专业水平。我们共同努力，顺利完成了这项研究。

最后，我要感谢我的家人和朋友们对我的学业和生活的支持和鼓励。他们的关心和鼓励让我能够全身心投入到研究中，并成为我继续前行的动力源泉。

